RKNN3 Supported Operators and Constraints Reference

Document ID: RK-YH-YF-418

Release Version: v1.2.0

Date: 2026-10-23

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. © 2026 Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, Software Boulevard, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

RKNN3 Supported Operators and Constraints Reference

Implementation path labels: `Subgraph`, `RegEmit`, `NRV`, `NRVReg`, and `Runtime/Simulator fallback`. The compiler selects the actual path based on the target platform, shapes, data types, and operator parameters. These labels do not replace the support scope and constraints of each operator.

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
v1.2.0	NPU Team	2026-10-23	Regenerated RK182X/RK3572 operator specifications, data types, and parameter constraints from current sources; added Atan, Tan, and exPagedAttention, and included the Cast and exDataConvert type-conversion operators.
v1.1.0	NPU Team	2026-08-14	1. Initial release of RK18X/RK3572 platform operator support and constraint reference

---

Table of Contents

- RKNN3 Supported Operators and Constraints Reference
  - RK182X
    - Convolution
      - Conv
      - ConvTranspose
      - ConvActivation
      - ConvRelu
      - ConvClip
      - ConvSigmoid
      - ConvTanh
      - ConvElu
      - ConvHardSigmoid
      - ConvHardSwish
      - ConvSwish
      - ConvSoftplus
      - ConvMish
      - ConvAdd
      - ConvAddRelu
      - ConvPRelu
      - ConvTransposeActivation
      - ConvTransposeRelu
      - ConvTransposeClip
      - ConvTransposeSigmoid

- ConvTransposeTanh
- ConvTransposeElu
- ConvTransposeHardSigmoid
- ConvTransposeHardSwish
- ConvTransposeSwish
- ConvTransposeSoftplus
- ConvTransposeMish
- ConvTransposeAdd
- ConvTransposePRelu
- exConvStreaming

Pooling

- AveragePool
- MaxPool
- GlobalAveragePool
- GlobalMaxPool

Elementwise Operations

- Add
- Sub
- Mul
- Div
- Min
- Max
- Pow
- Neg
- Exp
- Sqrt
- Reciprocal
- Floor
- Round
- Sin
- Cos
- Atan
- Tan
- exFastExp
- exTwoPow
- AddRelu
- MulRelu

Activation Functions
- Relu
- Sigmoid
- Clip
- LeakyRelu
- PRelu
- Elu
- Softplus
- HardSigmoid
- HardSwish
- Tanh
- Mish
- exGelu
- exHardSwish
- exMish
- exSwish
- exGlu
- exLogit

Comparison & Logical Operations
- Equal
- Greater
- GreaterOrEqual
- Less
- LessOrEqual
- Not
- And
- Where

Reduction Operations
- ReduceMax
- ReduceMean
- ReduceSum
- ArgMax
- ArgMin
- TopK

Tensor Transform
- Reshape

- Transpose
- Concat
- Split
- Expand
- Slice
- Pad
- DepthToSpace
- SpaceToDepth
- Resize
- exWindow

Index & Scatter

- Gather
- GatherElements
- GatherND
- ScatterND

Normalization

- BatchNormalization
- BatchNormRelu
- exNorm
- exSoftmax
- exSoftmaxMask

Matrix Operations

- exMatMul
- exMatMulActivation

LLM Extension Operators

- exAttention
- exPagedAttention
- exSDPAttention
- exLinearAttention
- exRoPE
- exSum

Type Conversion

- Cast
- exDataConvert

I/O Processing

- exInputProc

exOutputProc
RK3572
Convolution
Conv
ConvTranspose
ConvActivation
ConvRelu
ConvClip
ConvSigmoid
ConvTanh
ConvElu
ConvHardSigmoid
ConvHardSwish
ConvSwish
ConvSoftplus
ConvMish
ConvAdd
ConvAddRelu
ConvPRelu
ConvTransposeActivation
ConvTransposeRelu
ConvTransposeClip
ConvTransposeSigmoid
ConvTransposeTanh
ConvTransposeElu
ConvTransposeHardSigmoid
ConvTransposeHardSwish
ConvTransposeSwish
ConvTransposeSoftplus
ConvTransposeMish
ConvTransposeAdd
ConvTransposePRelu
exConvStreaming
Pooling
AveragePool
MaxPool

- GlobalAveragePool
- GlobalMaxPool

Elementwise Operations

- Add
- Sub
- Mul
- Div
- Min
- Max
- Pow
- Neg
- Exp
- Sqrt
- Reciprocal
- Floor
- Round
- Sin
- Cos
- Atan
- Tan
- exFastExp
- exTwoPow
- AddRelu
- MulRelu

Activation Functions

- Relu
- Sigmoid
- Clip
- LeakyRelu
- PRelu
- Elu
- Softplus
- HardSigmoid
- HardSwish
- Tanh
- Mish
- exGelu

- exHardSwish
- exMish
- exSwish
- exGlu
- exLogit

Comparison & Logical Operations

- Equal
- Greater
- GreaterOrEqual
- Less
- LessOrEqual
- Not
- And
- Where

Reduction Operations

- ReduceMax
- ReduceMean
- ReduceSum
- ArgMax
- ArgMin
- TopK

Tensor Transform

- Reshape
- Transpose
- Concat
- Split
- Expand
- Slice
- Pad
- DepthToSpace
- SpaceToDepth
- Resize
- exWindow

Index & Scatter

- Gather
- GatherElements

- GatherND
- ScatterND

Normalization

- BatchNormalization
- BatchNormRelu
- exNorm
- exSoftmax
- exSoftmaxMask

Matrix Operations

- exMatMul
- exMatMulActivation

LLM Extension Operators

- exAttention
- exPagedAttention
- exSDPAttention
- exLinearAttention
- exRoPE
- exSum

Type Conversion

- Cast
- exDataConvert

I/O Processing

- exInputProc
- exOutputProc

RK182X

Convolution

Conv

Convolution computation.

FORMULA

`output = Conv(input_tensor, weight, bias)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::_convolution`

INPUTS

- **input_tensor** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **weight** - `T1` (required):
  - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
  - Quantization: per-layer, per-channel
  - kernel_h range: [1, 31]
  - kernel_w range: [1, 31]
- **bias** - `T1` (optional):
  - Shape: C[output_channel]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **strides - int[stride_h, stride_w]**:

    - stride_h range: [1, 7]
    - stride_w range: [1, 7]
- **pads - int[pads_top, pads_left, pads_bottom, pads_right]**:
    - pads_top range: [0, 15]
    - pads_left range: [0, 15]
    - pads_bottom range: [0, 15]
    - pads_right range: [0, 15]
- **group - int**: —
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h range: [1, 31]
    - dilation_w range: [1, 31]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

SPECIAL CASES

Depthwise Convolution When Conv is used with depthwise convolution group semantics, the following differential specifications apply.

- **Input Differences — weight**: kernel_h range: [1, 8]; kernel_w range: [1, 8]
- **Attribute Differences — dilations**: dilation_h range: [1, 32]; dilation_w range: [1, 32]

ConvTranspose

Transposed convolution computation.

FORMULA

```
output = ConvTranspose(input_tensor, weight, bias)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::conv_transpose1d` , `aten::conv_transpose2d`

INPUTS

- **input_tensor** - `T1` (required):
    - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer
- When dilated_kernel_h > 1, input_tensor.width < 16383.
  The first layer input width must also satisfy the model input limitations.

- **weight** - `T1` (required):
  - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
  - Quantization: per-layer, per-channel
  - kernel_h range: [1, 31]
  - kernel_w range: [1, 31]
- **bias** - `T1` (optional):
  - Shape: C[output_channel]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **strides - int[stride_h, stride_w]**:
  - stride_h range: [1, 8]
  - stride_w range: [1, 8]
- **pads - int[pads_top, pads_left, pads_bottom, pads_right]**:
  - pads_top range: [0, 15]
  - pads_left range: [0, 15]
  - pads_bottom range: [0, 15]
  - pads_right range: [0, 15]
  - kernel_h * dilation_h - dilation_h - pads_top < 0 is not supported
    kernel_w * dilation_w - dilation_w - pads_left < 0 is not supported
    stride_h * (height - 1) - pads_top + 1 < output_h is not supported
    stride_w * (width - 1) - pads_left + 1 < output_w is not supported
- **group - int**:
  - Supports group = 1; when num_input = num_output, also supports group = num_output.
- **dilations - int[dilation_h, dilation_w]**:
  - dilation_h range: [1, 32]
  - dilation_w range: [1, 32]
  - When dilation != (1, 1), only kernel = (1, 1) is supported.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

ConvActivation

Fused Conv and generic Activation computation; activation type is determined by the Activation sub-operator.

FORMULA

```
output = Activation(Conv(input_tensor, weight, bias))
```

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

```
aten::_convolution
```

ConvRelu

Fused Conv and Relu computation.

FORMULA

```
output = Relu(Conv(input_tensor, weight, bias))
```

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

```
aten::_convolution
```

ConvClip

Fused Conv and Clip computation.

FORMULA

`output = Clip(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvSigmoid

Fused Conv and Sigmoid computation.

FORMULA

`output = Sigmoid(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvTanh

Fused Conv and Tanh computation.

FORMULA

`output = Tanh(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvElu

Fused Conv and Elu computation.

FORMULA

`output = Elu(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvHardSigmoid

Fused Conv and HardSigmoid computation.

FORMULA

`output = HardSigmoid(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvHardSwish

Fused Conv and HardSwish computation.

FORMULA

`output = HardSwish(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvSwish

Fused Conv and Swish computation.

FORMULA

`output = Swish(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvSoftplus

Fused Conv and Softplus computation.

FORMULA

`output = Softplus(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvMish

Fused Conv and Mish computation.

FORMULA

```
output = Mish(Conv(input_tensor, weight, bias))
```

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvAdd

Fused Conv and Add computation.

FORMULA

```
output = Add(Conv(input_tensor, weight, bias), residual)
```

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

EXTRA INPUTS

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

- Second input of Add.
  Supports broadcasting; broadcast constraints are the same as Add.

ConvAddRelu

Fused Conv, Add, and Relu computation.

FORMULA

`output = Relu(Add(Conv(input_tensor, weight, bias), residual))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

EXTRA INPUTS

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Second input of Add.
    Supports broadcasting; broadcast constraints are the same as Add.

ConvPRelu

Fused Conv and PRelu computation.

FORMULA

`output = PRelu(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvTransposeActivation

Fused ConvTranspose and generic Activation computation; activation type is determined by the Activation sub-operator.

FORMULA

```
output = Activation(ConvTranspose(input_tensor, weight, bias))
```

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeRelu

Fused ConvTranspose and Relu computation.

FORMULA

```
output = Relu(ConvTranspose(input_tensor, weight, bias))
```

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeClip

Fused ConvTranspose and Clip computation.

FORMULA

`output = Clip(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeSigmoid

Fused ConvTranspose and Sigmoid computation.

FORMULA

`output = Sigmoid(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeTanh

Fused ConvTranspose and Tanh computation.

FORMULA

`output = Tanh(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeElu

Fused ConvTranspose and Elu computation.

FORMULA

`output = Elu(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeHardSigmoid

Fused ConvTranspose and HardSigmoid computation.

FORMULA

`output = HardSigmoid(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeHardSwish

Fused ConvTranspose and HardSwish computation.

FORMULA

`output = HardSwish(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeSwish

Fused ConvTranspose and Swish computation.

FORMULA

`output = Swish(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeSoftplus

Fused ConvTranspose and Softplus computation.

FORMULA

`output = Softplus(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeMish

Fused ConvTranspose and Mish computation.

FORMULA

`output = Mish(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeAdd

Fused ConvTranspose and Add computation.

FORMULA

`output = Add(ConvTranspose(input_tensor, weight, bias), residual)`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

EXTRA INPUTS

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

- Second input of Add.
  Supports broadcasting; broadcast constraints are the same as Add.

ConvTransposePRelu

Fused ConvTranspose and PRelu computation.

FORMULA

`output = PRelu(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

---

exConvStreaming

Convolution computation with streaming cache.

FORMULA

`y = Conv(Concat(cache, x), w, b)`
`cache = Slice(Concat(cache, Slice(x, width=[0, tc])), width=[end - cache_w, end])`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::_convolution`

INPUTS

- **x** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **w** - `T1` (required):

    - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
    - Quantization: per-layer, per-channel
    - kernel_h range: [1, 31]
    - kernel_w range: [1, 31]
- **b** - `T1` (optional):
    - Shape: C[output_channel]
    - Quantization: per-layer
    - Optional; when there is no bias but tc needs to be passed, b can be an empty input placeholder.
- **tc** - `T2` (optional):
    - Shape: N[1]
    - tc indicates the effective width written to cache this time; the length must be less than the input width.
      When there are 3 inputs, if the size of the third input equals the output_channel of weight, it is treated as b; otherwise as tc.

OUTPUTS

- **y** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer

ATTRIBUTES

- **strides - int[stride_h, stride_w]**:
    - stride_h range: [1, 7]
    - stride_w range: [1, 7]
- **pads - int[pads_top, cache_w, pads_bottom, pads_right]**:
    - pads_top range: [0, 15]
    - pads_bottom range: [0, 15]
    - pads_right range: [0, 15]
    - cache_w is the streaming cache length; must be greater than 0.
- **group - int**: —
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h range: [1, 31]
    - dilation_w range: [1, 31]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

SPECIAL CASES

Depthwise exConvStreaming When exConvStreaming is used with depthwise convolution group semantics, the following differential specifications apply.

- **Input Differences** — **w**: kernel_h range: [1, 8]; kernel_w range: [1, 8]
- **Attribute Differences** — **dilations**: dilation_h range: [1, 32]; dilation_w range: [1, 32]

NOTES

Subgraph operator; the actual implementation is expanded by createSubGraph at the IR layer; emit only returns success.

Pooling

AveragePool

Performs average pooling on the input tensor with the specified kernel_shape, strides, and pads.

FORMULA

`Y = average_pool(X, kernel_shape, strides, pads)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::adaptive_avg_pool2d`, `aten::avg_pool2d`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - int8/int16 quantization requires height <= 32 and width <= 32

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **kernel_shape - int[kernel_h, kernel_w]**:
  - kernel_h range: [1, 32]
  - kernel_w range: [1, 32]
- **strides - int[stride_h, stride_w]**:
  - stride_h range: [1, 63]
  - stride_w range: [1, 63]
- **pads - int[pad_top, pad_left, pad_bottom, pad_right]**:
  - pad_top range: [0, 63]
  - pad_left range: [0, 63]
  - pad_bottom range: [0, 63]
  - pad_right range: [0, 63]
- **auto_pad - string**:
  - Only explicit pads are supported.
- **ceil_mode - int**:
  - Ceil mode is supported by adding bottom/right padding.
- **count_include_pad - int**:
  - When set to 1, the mean calculation includes the padding area.
- **dilations - int[dilation_h, dilation_w]**:
  - dilation_h range: [1, 1]
  - dilation_w range: [1, 1]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

MaxPool

Performs max pooling on the input tensor with the specified kernel_shape, strides, and pads.

FORMULA

`Y = max_pool(X, kernel_shape, strides, pads)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::max_pool1d`, `aten::max_pool2d`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **kernel_shape - int[kernel_h, kernel_w]**:
  - kernel_h range: [1, 32]
  - kernel_w range: [1, 256]
  - kernel_h * kernel_w <= 256
- **strides - int[stride_h, stride_w]**:
  - stride_h range: [1, 31]
  - stride_w range: [1, 256]
- **pads - int[pad_top, pad_left, pad_bottom, pad_right]**:
  - pad_top range: [0, 63]
  - pad_left range: [0, 63]
  - pad_bottom range: [0, 63]
  - pad_right range: [0, 63]
- **auto_pad - string**:
  - Only explicit pads are supported.
- **ceil_mode - int**:
  - Ceil mode is supported by adding bottom/right padding.
- **dilations - int[dilation_h, dilation_w]**:
  - dilation_h range: [1, 1]
  - dilation_w range: [1, 1]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int64>`):

GlobalAveragePool

Performs global average pooling on the H/W spatial dimensions of the input tensor.

FORMULA

```
Y = mean(X, axes=[2, 3])
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

```
aten::adaptive_avg_pool2d
```

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - int8 quantization requires height <= 32 and width <= 32;

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, 1, 1]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

GlobalMaxPool

Performs global max pooling on the spatial dimensions of the input tensor, outputting the maximum value per channel.

FORMULA

```
Y[batch, channel, 0, 0] = max(X[batch, channel, :, :])
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::adaptive_max_pool2d`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Hard constraint: height * width <= 256 and height <= 32.

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, 1, 1]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Elementwise Operations

Add

Performs element-wise binary addition, supporting NumPy-style broadcasting.

FORMULA

`C = A + B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::add` , `aten::add_`

INPUTS

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]

    - Quantization: per-layer
    - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1, T2`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer
    - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<int32>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Sub

Performs element-wise binary subtraction, supporting NumPy-style broadcasting.

FORMULA

`C = A - B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sub` , `aten::sub_`

INPUTS

- **A** - `T1, T2`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer
    - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1, T2`:

- Shape: NCHW[batch, channel, height, width]
- Quantization: per-layer
- Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Mul

Performs element-wise binary multiplication, supporting NumPy-style broadcasting.

FORMULA

`C = A * B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::mul` , `aten::mul_`

INPUTS

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Div

Performs element-wise binary division, supporting NumPy-style broadcasting.

FORMULA

`C = A / B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::div` , `aten::true_divide`

INPUTS

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`): **T2** in (`const_tensor<float16>`):

Min

Computes the element-wise minimum of multiple input tensors, supporting NumPy-style broadcasting.

FORMULA

`C = min(A, B, ...)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::min` , `aten::minimum`

INPUTS

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer, per-channel

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Max

Computes the element-wise maximum of multiple input tensors, supporting NumPy-style broadcasting.

FORMULA

`C = max(A, B, ...)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::max`, `aten::maximum`

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer, per-channel

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Pow

Performs element-wise exponentiation.

FORMULA

`output = input ^ exponent`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::pow`, `aten::pow_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **exponent** - `T2`:
  - Shape: S[scalar]

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Neg

Performs element-wise negation on the input tensor.

FORMULA

`Y = -X`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::neg`, `aten::neg_`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Exp

Computes the element-wise base-e exponential of the input tensor.

FORMULA

```
output = exp(input)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::exp` , `aten::exp_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Sqrt

Performs element-wise square root on the input tensor.

FORMULA

```
output = sqrt(input)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sqrt` , `aten::sqrt_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Reciprocal

Performs element-wise reciprocal on the input tensor.

FORMULA

```
Y = 1 / X
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::reciprocal`, `aten::reciprocal_`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Floor

Performs element-wise floor (rounds towards negative infinity) on the input tensor.

FORMULA

```
output = floor(input)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::floor`, `aten::floor_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Round

Rounding function using round-half-up semantics, formula: y = Floor(x + 0.5). Differs from ONNX Round's round-half-to-even (banker's rounding).

FORMULA

```
y = Floor(x + 0.5)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::round` , `aten::round_`

INPUTS

- **x** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`):

Sin

Computes the element-wise sine function.

FORMULA

```
y = sin(x)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sin`, `aten::sin_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

NOTES

Implementation first applies mod 2π to the input, then performs a LUT lookup.

Cos

Computes the element-wise cosine function.

FORMULA

`y = cos(x)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::cos`, `aten::cos_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

NOTES

Implementation first applies mod 2π to the input, then performs a LUT lookup.

Atan

Computes the elementwise arctangent.

FORMULA

```
y = atan(x)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::atan` , `aten::atan_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

NOTES

The main LUT covers x in [-16, 16]. Outside this range, interpolation uses the slope 1/(1+x^2), and the output converges to +/-pi/2.

Tan

Computes the elementwise tangent.

FORMULA

`y = tan(x)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::tan`, `aten::tan_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`, `tensor<bfloat16>`):

NOTES

The implementation first applies mod pi to the input (result in [-pi/2, pi/2]), then looks up the LUT. The main LUT only covers [-1, 1]; values outside this range are clamped to tan(+/-1). Accuracy is therefore not guaranteed for inputs whose mod-pi result lies outside [-1, 1].

---

exFastExp

Computes the element-wise natural exponential using an approximation for improved performance.

FORMULA

`output ≈ exp(input)`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::exp`, `aten::exp_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

---

exTwoPow

Computes the element-wise power of 2.

FORMULA

`y = 2^x`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::pow`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

AddRelu

Fused Add and Relu computation.

FORMULA

```
C = Relu(A + B)
```

INHERITANCE

Inherits from Add.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::add` , `aten::add_`

MulRelu

Fused Mul and Relu computation.

FORMULA

```
C = Relu(A * B)
```

INHERITANCE

Inherits from Mul.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::mul`, `aten::mul_`

Activation Functions

Relu

Rectified Linear Unit, performs max(input, 0) element-wise.

FORMULA

```
output = max(input, 0)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::relu`, `aten::relu6`, `aten::relu_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Sigmoid

Applies the Sigmoid function element-wise to the given input tensor.

FORMULA

```
y = 1 / (1 + exp(-x))
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sigmoid`, `aten::sigmoid_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Clip

Clips the input tensor elements to the [min, max] range; out-of-bounds values are clamped to the boundary.

FORMULA

```
output = Clip(input, min, max)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::clamp`, `aten::clamp_`

INPUTS

- **input** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **min** - `T2` (optional):

- Shape: S[scalar]
- Clip lower bound, scalar. Omitted means no lower bound.

- **max** - `T2` (optional):
  - Shape: S[scalar]
  - Clip upper bound, scalar. Omitted means no upper bound.

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

LeakyRelu

Leaky Rectified Linear Unit; positive values remain unchanged, negative values are scaled by a fixed ratio alpha.

FORMULA

```
output = input if input >= 0 else alpha * input
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::leaky_relu` , `aten::leaky_relu_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **alpha - float[alpha]**:
  - alpha range: [0, 1]
  - Negative value scaling factor.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

PRelu

Parametric Rectified Linear Unit; positive values remain unchanged, negative values are scaled by a learnable parameter slope.

FORMULA

```
output = input if input >= 0 else slope * input
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::prelu` , `aten::prelu_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer, per-channel
- **slope** - `T2` :
  - Shape: C[channel]
  - Quantization: per-channel
  - Supports a single scalar or C-dimension coefficients.

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer, per-channel

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Elu

Applies the Elu function element-wise, alpha = 1.

FORMULA

`f(x) = alpha * (exp(x) - 1) for x < 0; f(x) = x for x >= 0`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::elu` , `aten::elu_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Softplus

Applies the Softplus function element-wise.

FORMULA

`y = ln(exp(x) + 1)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::softplus`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

HardSigmoid

Applies the HardSigmoid function element-wise, controlled by alpha and beta attributes.

FORMULA

```
y = max(0, min(1, alpha * x + beta))
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::hardsigmoid` , `aten::hardsigmoid_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

ATTRIBUTES

- **alpha - float**:
  - Default 0.2 (consistent with the ONNX standard).
- **beta - float**:
  - Default 0.5.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

HardSwish

Applies the HardSwish function element-wise, alpha = 1/6, beta = 0.5.

FORMULA

```
y = x * max(0, min(1, alpha * x + beta))
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::hardswish` , `aten::hardswish_`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Tanh

Computes the element-wise hyperbolic tangent of the given input tensor.

FORMULA

`y = tanh(x)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::tanh` , `aten::tanh_`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Mish

Applies the Mish function element-wise.

FORMULA

`y = x * tanh(softplus(x)) = x * tanh(ln(1 + exp(x)))`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::mish`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exGelu

Applies the Gelu function element-wise.

FORMULA

`y = x * Phi(x), when Phi uses tanh approximation: y = 0.5 * x * (1 + tanh(sqrt(2/pi) * (x + 0.044715 * x^3)))`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::gelu`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exHardSwish

Applies the HardSwish function element-wise, alpha = 1/6, beta = 0.5.

FORMULA

```
y = x * max(0, min(1, alpha * x + beta))
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::hardswish` , `aten::hardswish_`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exMish

Applies the Mish function element-wise.

FORMULA

```
y = x * tanh(softplus(x)) = x * tanh(ln(1 + exp(x)))
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::mish`, `aten::mish_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

---

exSwish

Applies the Swish function (also known as SiLU) element-wise.

FORMULA

```
y = x * sigmoid(x)
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::silu`, `aten::silu_`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exGlu

Splits the input tensor into two equal parts along the specified axis, applies sigmoid gating to the second part, and multiplies it with the first part.

FORMULA

```
a, b = np.split(X, 2, axis=axis)
Y = a * sigmoid(b)
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::glu`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - The length of the dimension specified by axis must be even.
    The NPU subgraph implementation primarily targets axis = 1 channel dimension splitting.

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, out_channel, height, width]
  - Quantization: per-layer
  - The output shape is consistent with the input shape, but the dimension specified by axis becomes half of the input.
    When axis = 1, out_channel = channel / 2.

ATTRIBUTES

- **axis - int**:

- Specifies the dimension for GLU split; valid range is [-rank, rank - 1].
  Negative axes are converted to positive axes following NumPy semantics.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exLogit

Applies the logit activation function to the input tensor.

FORMULA

```
Y = log(X / (1 - X))
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::logit`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Comparison & Logical Operations

Equal

Compares whether two tensors are equal element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

`C = (A == B)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::eq`, `aten::eq_`

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Boolean semantic result; RKNN stores it in int8 memory layout.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`tensor<bool>`, `tensor<int8>`):

Greater

Compares whether A is greater than B element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

`C = (A > B)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::gt`, `aten::gt_`

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.

OUTPUTS

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

GreaterOrEqual

Compares whether A is greater than or equal to B element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

```
C = (A >= B)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::ge` , `aten::ge_`

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

OUTPUTS

- **C** - `T2` :
  - Shape: NCHW[batch, channel, height, width]
  - Boolean semantic result; RKNN stores it in int8 memory layout.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Less

Compares whether A is less than B element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

`C = (A < B)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::lt` , `aten::lt_`

INPUTS

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.
- **B** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.

OUTPUTS

- **C** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

LessOrEqual

Compares whether A is less than or equal to B element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

`C = (A <= B)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::le` , `aten::le_`

INPUTS

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.
- **B** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.

OUTPUTS

- **C** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Not

Performs element-wise logical NOT on the input tensor.

FORMULA

`Y = !X`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::logical_not`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>`):

And

Performs element-wise logical AND, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

`C = A && B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::bitwise_and`, `aten::logical_and`

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>` , `tensor<int8>`):

Where

Selects elements from x or y based on the condition, supporting NumPy-style broadcasting.

FORMULA

```
output = select(condition, x, y)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::where`

INPUTS

- **condition** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
- **x** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.
- **y** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>`): **T2** in (`tensor<int8>` , `tensor<float16>`):

Reduction Operations

ReduceMax

Computes the maximum of the input tensor along the specified axis, supporting multiple axes and keepdims.

FORMULA

```
output = ReduceMax(input, axes, keepdims)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::amax` , `aten::max`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - When keepdims=1, the reduced axis has size 1; when keepdims=0, the axis is removed from the output. The actual output rank depends on axes and keepdims.

ATTRIBUTES

- **axes - int[]**:

- Axis for computing the maximum value.

- **keepdims - int**:
  - Whether to keep the reduced dimensions (1 keeps, 0 removes).

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

ReduceMean

Reduces the input tensor by computing the mean along the specified axes.

FORMULA

`Y = mean(data, axes)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::mean`

INPUTS

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - For int8/int16 quantization with axes = [2], [3], [2, 3], or
    [1, 2, 3], height <= 32 and
    width <= 32 are required
- **axes** - `T2` :
  - Shape: Axes[num_axes]
  - Specifies the dimensions to reduce; negative axes are converted to positive axes following NumPy semantics.
    Supports axes = [1], [2], [3], [2, 3], [1, 2, 3]

OUTPUTS

- **reduced** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`const_tensor<int64>`):

ReduceSum

Reduces the input tensor by computing the sum along the specified axes.

FORMULA

`Y = sum(data, axes)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sum`

INPUTS

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - For int8/int16 quantization with axes = [2], [3], [2, 3], or [1, 2, 3], height <= 32 and width <= 32 are required
- **axes** - `T2` :
  - Shape: Axes[num_axes]
  - Specifies the dimensions to reduce; negative axes are converted to positive axes following NumPy semantics.
    Supports axes = [1], [2], [3], [2, 3], [1, 2, 3]

OUTPUTS

- **reduced** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`const_tensor<int64>`):

ArgMax

Returns the indices of the maximum values along the specified axis.

FORMULA

`output = ArgMax(input, axis)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::argmax`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **axis - int**:
  - Specifies the axis for argmax operation; valid range [0, ndim-1].
- **keepdims - int**:
  - Whether to keep the output dimensions the same as the input. 0 means no, 1 means yes.
- **select_last_index - int**:
  - When multiple maximum values exist, whether to select the last index. 0 selects the first, 1 selects the last.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

ArgMin

Returns the indices of the minimum values along the specified axis.

FORMULA

`output = ArgMin(input, axis)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::argmin`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **axis - int**:
  - Specifies the axis for argmin operation; valid range [0, ndim-1].
- **keepdims - int**:
  - Whether to keep the output dimensions the same as the input. 0 means no, 1 means yes.
- **select_last_index - int**:
  - When multiple minimum values exist, whether to select the last index. 0 selects the first, 1 selects the last.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

TopK

Retrieves the K largest (or smallest, determined by largest) values and their indices along the specified axis.

FORMULA

`(values, indices) = TopK(X, K, axis, largest, sorted)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::topk`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **K** - `T3` :
  - Shape: S[scalar]
  - The top K values, scalar, stored in an int64 tensor.

OUTPUTS

- **values** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - The top K values.
- **indices** - `T2` :
  - Shape: NCHW[batch, channel, height, width]
  - Index of the corresponding value. The ONNX standard is int64; RKNN actually outputs int32 or int64.

ATTRIBUTES

- **axis - int**:
  - Axis for computation; default -1.
- **largest - int**:
  - 1 selects maximum, 0 selects minimum, default 1.
- **sorted - int**:
  - 1 outputs in descending order, 0 does not guarantee order, default 1.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>` , `tensor<int64>`): **T3** in (`tensor<int64>`):

Tensor Transform

Reshape

Returns a tensor with the specified shape without changing the input data or element count.

FORMULA

`output = reshape(input, shape)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::reshape`, `aten::reshape_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **shape - int**:
  - Specifies the output tensor shape.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Transpose

Transposes the input tensor according to the axis order specified by perm.

FORMULA

`Y = transpose(X, perm)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::permute`, `aten::transpose`, `aten::transpose_`

INPUTS

- **X** - `T1, T2`:
  - Shape: NCHW[n1, c1, h1, w1]
  - Quantization: per-layer
  - For detailed constraint rules, see the perm attribute list.

OUTPUTS

- **Y** - `T1, T2`:
  - Shape: NCHW[n2, c2, h2, w2]
  - Quantization: per-layer
  - For detailed constraint rules, see the perm attribute list.

ATTRIBUTES

- **perm - int[d0, d1, d2, d3]**:
  - Axis order for transpose. Constraints for different perm parameters are listed in the perm constraint table below.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`): **T2** in (`tensor<int16>`, `tensor<float16>`):

NOTES

Different perm parameter constraints are as follows (where p = 268435456, q = 65536):

perm	T1 Constraint	T2 Constraint
[0,1,2,3]	No restriction	No restriction
[0,1,3,2]	h1%16=0, h1*w1*<*16*p, h1*w1*<*2*q	h1%8=0, h1*w1*<*8*p, h1*w1*<*2*q
[0,2,3,1]	c1<8192, h1*w1%8=0, w1*c1<p*16	c1<8192, h1*w1%8=0, w1*c1<p*8
[0,2,1,3]	h1%16=0, h1*w1*<*16*p, h1*w1*<*2*q, c1*w1<p	h1%8=0, h1*w1*<*8*p, h1*w1*<*2*q, c1*w1<p
[0,3,1,2]	h1*w1*<*p, w1*<*8192, h1*c1<p	h1*w1*<*p, w1*<*8192, h1*c1<p
[0,3,2,1]	h1%16=0, h1*w1*<*16*p, h1*w1*<*2*q, c1<p*16	h1%8=0, h1*w1*<*8*p, h1*w1*<*2*q, c1<p*8
[1,0,2,3]	h1*w1*<*16*p, n1<8192, h1*w1<p	h1*w1*<*8*p, n1<8192, h1*w1<p

perm	T1 Constraint	T2 Constraint
[1,0,3,2]	h1%16=0, c1*h1*w1<16*p, h1*w1<2*q	h1%8=0, c1*h1*w1<8*p, h1*w1<2*q
[1,2,0,3]	n1*w1*<*p*16, c1*h1*w1<p, w1%16=0	n1*w1*<*p*8, c1*h1*w1<p, w1%8=0
[1,2,3,0]	h1*w1*<*p, n1*<*p*16, c1*h1*w1<p*16, n1*w1<p*16	h1*w1*<*p, n1*<*p*8, c1*h1*w1<p*8, n1*w1<p*8
[1,3,0,2]	c1*w1*<*8192, n1*h1<p, w1%16=0	c1*w1*<*8192, n1*h1<p, w1%8=0
[1,3,2,0]	c1*h1*w1<p, n1*h1<p, w1<8192, w1%16=0	c1*h1*w1<p, n1*h1<p, w1<8192, w1%8=0
[2,1,0,3]	c1<8192, c1*h1*w1<p, n1*w1<p	c1<8192, c1*h1*w1<p, n1*w1<p
[2,1,3,0]	c1*h1*w1<p, n1*w1*<*p*16, c1<8192, n1*w1%4=0	c1*h1*w1<p, n1*w1*<*p*8, c1<8192, n1*w1%4=0
[2,0,3,1]	c1<8192, c1*h1*w1<p	c1<8192, c1*h1*w1<p
[2,0,1,3]	n1*c1*h1*w1*<*p, h1*w1%16=0	n1*c1*h1*w1*<*p, h1*w1%8=0
[2,3,1,0]	c1*h1*w1<p, n1*c1*<*p, h1*w1<8192, h1*w1%16=0	c1*h1*w1<p, n1*c1*<*p, h1*w1<8192, h1*w1%8=0
[2,3,0,1]	h1*w1*<*8192, n1*c1<p	h1*w1*<*8192, n1*c1<p
[3,0,2,1]	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*w1%16=0	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*w1%8=0
[3,0,1,2]	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*h1*w1%16=0	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*h1*w1%8=0
[3,2,0,1]	h1*w1*<*p, n1*c1<8192, n1*c1*w1<p	h1*w1*<*p, n1*c1<8192, n1*c1*w1<p
[3,2,1,0]	c1*h1*w1<p, h1*w1*<*8192, 2*h1*w1*<*q, n1*c1<p*16, h1%16=0	c1*h1*w1<p, h1*w1*<*8192, 2*h1*w1*<*q, n1*c1<p*8, h1%8=0
[3,1,0,2]	h1*w1*<*p, c1*w1<8192, n1*h1<p	h1*w1*<*p, c1*w1<8192, n1*h1<p
[3,1,2,0]	h1*w1*<*p, n1*c1<8192, n1*h1*<*p, n1*c1*w1<p	h1*w1*<*p, n1*c1<8192, n1*h1*<*p, n1*c1*w1<p

---

Concat

Concatenates multiple tensors along the specified axis into a single tensor; all inputs must have the same size except along the axis dimension.

FORMULA

`output = Concat(input_tensor_list, axis)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::cat`

INPUTS

- **input_tensor** - `T` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Input is a list of tensors; all tensors must have the same size except along the axis dimension.

OUTPUTS

- **output** - `T` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **axis - int[axis]**:
  - axis range: [-4, 3]
  - Concatenation axis; RKNN requires 4D NCHW input, supports negative axes.

DATA TYPE CONSTRAINTS

T in (`tensor<int8>` , `tensor<float16>`):

Split

Splits the input tensor into multiple output tensors along the specified axis.

FORMULA

```
outputs = Split(input_tensor, axis, num_outputs, split)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::split`

INPUTS

- **input_tensor** - `T` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - One or more output tensors; the count is determined by num_outputs, and the shape depends on the split.

ATTRIBUTES

- **axis - int[axis]**:
  - axis range: [-4, 3]
  - Split axis; RKNN requires 4D NCHW input, supports negative axes.
- **num_outputs - int**: —
- **split - int[]**:
  - Specifies the size of each output along the axis dimension after splitting.

DATA TYPE CONSTRAINTS

T in (`tensor<int8>` , `tensor<float16>`):

Expand

Broadcasts the input tensor to the target shape.

FORMULA

```
output = Expand(input, target_shape)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::expand` , `aten::expand_as`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Slice

Obtains a slice of the input tensor following NumPy slicing rules.

FORMULA

```
output = Slice(data, starts, ends, axes, steps)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::narrow` , `aten::slice`

INPUTS

- **data** - `T` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **starts** - `T2` (required):
  - Shape: D[dim]
  - Per-axis split start positions.
- **ends** - `T2` (required):
  - Shape: D[dim]
  - Per-axis split end positions.
- **axes** - `T2` (optional):
  - Shape: D[dim]
  - Selects the split axis; supports axes 0~3, supports multiple axes simultaneously; when omitted, all axes are processed.
- **steps** - `T2` (optional):
  - Shape: D[dim]

- Per-axis split step sizes. When height*width==1 and channel%subc==0 and starts[1]%subc==0 and ends[1]%subc==0 and steps[1]<=subc, steps can be non-1; otherwise, the NPU can only run when steps==[1,1,1,1].

OUTPUTS

- **output** - `T`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int64>`):

Pad

Pads the input tensor with the specified pads and mode.

FORMULA

```
output = Pad(data, pads, mode, constant_value)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::constant_pad_nd`

INPUTS

- **data** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **pads** - `T2` (required):
  - Shape: PADS[batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end, height_end, width_end]
- **constant_value** - `T1` (optional):
  - Shape: S[scalar]
  - Quantization: per-layer
  - Same type as data; only effective in constant mode, defaults to 0.

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **mode - string**:
  - constant mode has no range restrictions; edge mode only supports h/w directions.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`tensor<int64>`):

SPECIAL CASES

Reflect Pad In reflect mode, input shape components have additional range restrictions.

- **Input Differences** — **data**: channel range: [1, 8192]; height range: [1, 8192]; width range: [1, 8176]

DepthToSpace

Rearranges the channel dimension of the input tensor back to H/W spatial dimensions by blocksize.

FORMULA

```
N, C, H, W = X.shape
b = blocksize
out_channel = C // (b * b)
Y = np.empty((N, out_channel, H * b, W * b), dtype=X.dtype)

for n in range(N):
    for c in range(out_channel):
        for h in range(H * b):
            for w in range(W * b):
                offset = (h % b) * b + (w % b)
                in_c = c + offset * out_channel
                Y[n, c, h, w] = X[n, in_c, h // b, w // b]
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::pixel_shuffle`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - channel must be divisible by blocksize * blocksize.

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, out_channel, out_height, out_width]
  - Quantization: per-layer
  - out_channel = channel / (blocksize * blocksize)。
    out_height = height * blocksize。
    out_width = width * blocksize。

ATTRIBUTES

- **blocksize - int**:
  - Spatial block size; must be greater than 0.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>`):

SpaceToDepth

Rearranges H/W spatial blocks of the input tensor to the channel dimension.

FORMULA

```
N, C, H, W = X.shape
b = blocksize
Y = np.empty((N, C * b * b, H // b, W // b), dtype=X.dtype)

for n in range(N):
    for c in range(C):
        for h in range(H):
            for w in range(W):
                out_c = c + ((h % b) * b + (w % b)) * C
                Y[n, out_c, h // b, w // b] = X[n, c, h, w]
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::pixel_unshuffle`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - height and width must be divisible by blocksize.

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, out_channel, out_height, out_width]
  - Quantization: per-layer
  - out_channel = channel * blocksize * blocksize。
    out_height = height / blocksize。
    out_width = width / blocksize。

ATTRIBUTES

- **blocksize - int**:
  - Spatial block size; must be greater than 0.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>`):

Resize

Resizes the spatial dimensions of the input tensor according to the specified scales or sizes, supporting nearest and linear interpolation modes.

FORMULA

`Y = Resize(X, scales, mode, coordinate_transformation_mode, nearest_mode)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::upsample_bilinear2d`, `aten::upsample_nearest2d`

INPUTS

- **X** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **roi** - `T3` (optional):
  - Shape: N[8]
  - Crop region coordinates [start_x, start_y, start_z, start_t, end_x, end_y, end_z, end_t]; only the default value [0,0,0,0,1,1,1,1] is supported.
- **scales** - `T3` (optional):
  - Shape: N[4]
  - Per-dimension scale factors [batch_scale, channel_scale, height_scale, width_scale], mutually exclusive with sizes. In linear and
    nearest modes, only integer scaling is supported, and
    batch/channel dimension scale must be 1, H/W dimension scale <= 8.
- **sizes** - `T4` (optional):
  - Shape: N[4]
  - Target output size [batch, channel, height, width], mutually exclusive with scales. In linear and
    nearest modes, only integer scaling is supported, batch/channel dimensions remain unchanged, and
    H/W dimension multiplier <= 8.

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, out_height, out_width]
  - Quantization: per-layer

ATTRIBUTES

- **mode - string**:
  - linear and nearest modes are internally implemented via convolution simulation.
- **coordinate_transformation_mode - string**:
  - tf_crop_and_resize is not supported.
- **cubic_coeff_a - float**:
  - Cubic mode coefficient; currently only supports the default value -0.75.

- **exclude_outside - int**:
  - Excluding outer points is not supported.
- **extrapolation_value - float**:
  - Extrapolation value setting is not supported.
- **nearest_mode - string**:
  - Rounding method in nearest mode.
- **antialias - int**:
  - Antialiasing is not supported.
- **keep_aspect_ratio_policy - string**:
  - Only the default stretch mode is supported.
- **axes - int[axis_0, axis_1, axis_2, axis_3]**:
  - Specifies the scaling dimension; the current implementation does not impose additional constraints on this attribute.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T3** in (`tensor<float32>`): **T4** in (`tensor<int64>`):

---

exWindow

exWindow is derived from the window_partition and window_reverse operations in https://github.com/microsoft/Swin-Transformer/blob/main/models/swin_transformer.py . In the original paper, the input feature is divided into multiple non-overlapping windows, each of size window_size × window_size. Both input and output features must be 4D shapes, and division is performed along both height and width dimensions.

FORMULA

```
B, C, H, W = X.shape

if mode == "partition":
    x = X.reshape(B, C, num_windows[0], window_sizes[0],
                  num_windows[1], window_sizes[1])
    y = np.transpose(x, (0, 1, 2, 4, 3, 5))
    Y = y.reshape(B, C,
                  num_windows[0] * num_windows[1],
                  window_sizes[0] * window_sizes[1])

elif mode == "reverse":
    x = X.reshape(B, C, num_windows[0], num_windows[1],
                  window_sizes[0], window_sizes[1])
    y = np.transpose(x, (0, 1, 2, 4, 3, 5))
    Y = y.reshape(B, C,
                  num_windows[0] * window_sizes[0],
                  num_windows[1] * window_sizes[1])

elif mode == "partition_num_first":
    x = X.reshape(B, C, window_sizes[0], num_windows[0],
                  window_sizes[1], num_windows[1])
    y = np.transpose(x, (0, 1, 3, 5, 2, 4))
    Y = y.reshape(B, C,
                  num_windows[0] * num_windows[1],
                  window_sizes[0] * window_sizes[1])
```

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - partition requires height = num_h * win_h, width = num_w * win_w.
    reverse requires height = num_h * num_w, width = win_h * win_w.
    partition_num_first requires height = win_h * num_h, width = win_w * num_w.

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, out_height, out_width]
  - Quantization: per-layer
  - partition and partition_num_first output out_height = num_h * num_w, out_width = win_h * win_w.
    reverse outputs out_height = num_h * win_h, out_width = num_w * win_w.

ATTRIBUTES

- **mode - string**:
  - partition follows the window_partition implementation.
    reverse follows the window_reverse implementation.
    The difference between partition_num_first and partition is that partition_num_first divides H/W by num_windows, while partition divides H/W by window_sizes.
- **num_windows - int[num_h, num_w]**:
  - num_h range: [1, 8192]
  - num_w range: [1, 8192]
  - 2D vector, number of windows. Defines how many windows the height and width of the input feature are divided into.
- **window_sizes - int[win_h, win_w]**:
  - win_h range: [1, 8192]
  - win_w range: [1, 8192]
  - 2D vector, window size. Defines the height and width of each window. Attention computation and feature extraction within the window are based on this size.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Index & Scatter

Gather

Gathers elements from the input tensor along the specified axis based on indices.

FORMULA

```
output = Gather(input, indices, axis)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::embedding` , `aten::gather`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

    - Quantization: per-layer
- **indices** - `T2`:
    - Shape: INDICES[batch, index]

OUTPUTS

- **output** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer

ATTRIBUTES

- **axis - int**:
    - Gathers elements along this axis.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

GatherElements

Gathers elements from the input tensor along the specified axis by indices; the output shape matches that of indices.

FORMULA

```
output = GatherElements(data, indices, axis)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::gather`

INPUTS

- **data** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer
- **indices** - `T2`:
    - Shape: NCHW[batch, channel, height, width]
    - Supports both constant and non-constant indices.

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **axis - int**:
  - Gathers elements along this axis; negative values are normalized to the valid range.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<int32>`): **T2** in (`tensor<int32>`):

GatherND

Gathers slices from the input tensor based on indices and combines them into a new output tensor.

FORMULA

```
output = GatherND(data, indices, batch_dims)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::gather_nd`

INPUTS

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **indices** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **batch_dims - int**:
  - Specifies how many leading dimensions are treated as batch dimensions; no gather operation is performed on these dimensions.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

ScatterND

Scatters updates into the data tensor at positions specified by indices to produce an output tensor, supporting reduction modes.

FORMULA

```
output = ScatterND(data, indices, updates, reduction)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::scatter_nd`

INPUTS

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **indices** - `T2` :
  - Shape: INDICES[num_index, index_dim]
- **updates** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **reduction - string**:
  - Specifies the reduction mode for scatter.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>` , `tensor<int64>`):

Normalization

BatchNormalization

Performs batch normalization on the input tensor (ref: https://arxiv.org/abs/1502.03167).

FORMULA

`output = (input - mean) / sqrt(variance + epsilon)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::batch_norm`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **scale** - `T2` :
  - Shape: C[channel]
  - Quantization: per-channel
- **B** - `T2` :
  - Shape: C[channel]
  - Quantization: per-channel
- **mean** - `T2` :
  - Shape: C[channel]
  - Quantization: per-channel
  - During inference, it is folded into scale/B together with var and does not participate in computation at runtime.
- **var** - `T2` :

- Shape: C[channel]
- Quantization: per-channel
- During inference, it is folded into scale/B together with mean and does not participate in computation at runtime.

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **epsilon - float**:
  - A small real number added to the standard deviation to prevent division by zero; default 1e-5.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

BatchNormRelu

Fused BatchNormalization and Relu computation.

FORMULA

```
output = Relu(BatchNormalization(input))
```

INHERITANCE

Inherits from BatchNormalization.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::batch_norm`

---

exNorm

Performs extended normalization computation, with optional gamma scaling and beta shift.

FORMULA

```
x = X
weight = W if W is not None else 1
bias = B if B is not None else 0

if mode == "layer":
    x_centered = x - np.mean(x, axis=axes, keepdims=True)
    y = x_centered * fast_inv_sqrt(
        np.mean(x_centered * x_centered, axis=axes, keepdims=True) + eps)

elif mode == "mean_var":
    mean = np.mean(x, axis=axes, keepdims=True)
    var = np.mean(x * x, axis=axes, keepdims=True) - mean * mean
    y = (x - mean) * fast_inv_sqrt(var + eps)

elif mode == "rms":
    y = x * fast_inv_sqrt(
        np.mean(x * x, axis=axes, keepdims=True) + eps)

elif mode == "pixel":
    y = x * fast_inv_sqrt(
        np.sum(x * x, axis=axes, keepdims=True) + eps)

elif mode == "l1":
    y = x / np.maximum(
        np.sum(np.abs(x), axis=axes, keepdims=True), eps)

elif mode == "l2":
    y = x / np.maximum(
        fast_sqrt(np.sum(x * x, axis=axes, keepdims=True)), eps)

elif mode == "rms_reciprocal":
    y = fast_inv_sqrt(
        np.mean(x * x, axis=axes, keepdims=True) + eps)

elif mode == "gaussian_topk":
    mean = np.mean(x, axis=axes, keepdims=True)
    std = fast_sqrt(np.mean(x * x, axis=axes, keepdims=True) - mean * mean)
    cutoff_x = mean + std * std_multipler
    y = np.clip(x - cutoff_x, a_min=0, a_max=None).astype(x.dtype)

Y = y * weight + bias
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::layer_norm`, `aten::rms_norm`

INPUTS

- **X** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]

    - Quantization: per-layer
- **W** - `T1` (optional):
    - Shape: NCHW[batch, channel, height, width], [1]
    - Quantization: per-layer
    - Gamma scale parameter; supports per-layer, per-channel, or broadcast shapes compatible with the output.
- **B** - `T1` (optional):
    - Shape: NCHW[batch, channel, height, width], [1]
    - Quantization: per-layer
    - Beta shift parameter; supports per-layer, per-channel, or broadcast shapes compatible with the output.

OUTPUTS

- **Y** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer
    - When mode is rms_reciprocal, the dimensions specified by axes maintain keepdims semantics and output as 1.

ATTRIBUTES

- **mode - string**:
    - Valid values: layer, mean_var, rms, pixel, l1, l2, rms_reciprocal, gaussian_topk.
- **axes - int[axis0, axis1, axis2]**:
    - Default value is [1].
      Shape inference supports negative axes; the NPU subgraph implementation primarily supports [1] and [1, 2, 3].
- **eps - float**:
    - Default value 1e-5, used to avoid division by zero.
- **std_multipler - float**:
    - Only used by gaussian_topk; default value is 1.0.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

---

exSoftmax

Applies softmax normalization along the specified axis of the input; the output shape is the same as the input.

FORMULA

`exSoftmax(input, axis) = exp(input - max(input, axis)) / sum_axis(exp(input - max(input, axis)))`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::softmax`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Subject to the constraints of transpose specifications.

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **axis - int**:
  - The axis for softmax; 1 is the channel direction, 3 is the width direction. Supporting only axis 1 and 3 is an RKNN platform limitation; the ONNX standard allows any valid axis.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exSoftmaxMask

Masked Softmax computation: sets masked positions to -inf using mask_value, then applies Softmax.

FORMULA

`input(x, mask_t, mask_value) = x + (mask_t - 1) * inf, if mask_value = 0`
`x + mask_t * (-inf), if mask_value = 1`

`exSoftmaxMask(input, axis) = exp(input - max(input, axis)) / sum_axis(exp(input - max(input, axis)))`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::softmax`

INPUTS

- **input** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Subject to the constraints of transpose specifications.
- **mask** - `T1` (optional):
  - Shape: NCHW[1, channel, height, width]
  - channel range: [1, 8192]
  - width range: [1, 8192]
  - When axis=1, width has no restriction; when axis=3/-1, width range is [1, 8192].

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **axis - int**:
  - The axis for softmax; 1 is the channel direction, 3 is the width direction. Supporting only axis 1 and 3 is an RKNN platform limitation; the ONNX standard allows any valid axis.
- **mask_value - int**:
  - 0 indicates positions to keep, 1 indicates positions to mask.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Matrix Operations

exMatMul

Matrix multiplication operation.

FORMULA

`Y = A · B + C, where C is bias (bias is optional)`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::matmul`, `aten::bmm`, `aten::mm`

INPUTS

- **A** - `T1` (required):
  - Shape: NK1N[batch, k, 1, n]
  - Quantization: per-layer、per-channel、group32/64/128
- **B** - `T1` (required):
  - Shape: NK1M[batch, k, 1, m]
  - Quantization: per-layer
- **C** - `T1, T2` (optional):
  - Shape: NM1M[batch, n, 1, m]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **Y** - `T1`:
  - Shape: NM1M[batch, n, 1, m]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`const_tensor<float16>`):

---

exMatMulActivation

Fused exMatMul and generic Activation computation; activation type is determined by the Activation sub-operator.

FORMULA

`output = Activation(exMatMul(input, weight, bias))`

INHERITANCE

Inherits from exMatMul.

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::matmul`, `aten::bmm`, `aten::mm`

LLM Extension Operators

exAttention

General attention subgraph operator supporting FullAttention and SlidingAttention modes.

FORMULA

`output = Matmul(Softmax(Matmul(rope(query, cos_pos_emb, sin_pos_emb), rope(key, cos_pos_emb, sin_pos_emb)) + attention_mask), value)`
`output = Matmul(Softmax(Matmul(query, key) + attention_mask), value)`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::multi_head_attention_forward`

INPUTS

- **query** - `T1` (required):
  - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]
- **key** - `T1` (required):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **value** - `T1` (required):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **attention_mask** - `T1` (required):
  - Shape: NCHW[1, seq_len, 1, seq_len]
- **cos_pos_emb** - `T1` (optional):

    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
- **sin_pos_emb** - `T1` (optional):
    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
- **Th** - `T2` (required):
    - Shape: N[1]
    - history token len; the number of tokens previously processed.
- **Tc** - `T2` (required):
    - Shape: N[1]
    - current token len; the number of valid tokens in the current inference bucket.
- **Ts** - `T2` (required):
    - Shape: N[1]
    - system prompt token len; the aligned system prompt length. Both [182X, 3572] require alignment to a multiple of 32.
- **Tsr** - `T2` (required):
    - Shape: N[1]
    - Real system prompt token len; the actual system prompt length, less than or equal to Ts.

OUTPUTS

- **output** - `T1`:
    - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]

ATTRIBUTES

- **attention_type - string**:
    - Represents global attention and sliding window attention modes respectively. Valid values: [FullAttention, SlidingAttention], default FullAttention.
- **rope - int**:
    - Whether to use RoPE positional encoding; 0 means not used, 1 means used, default 0.
- **kvcache_store_method - string**:
    - Indicates the KV cache storage method; Normal stores float16 data directly, GroupQuant stores with group quantization. Valid values: [Normal, GroupQuant], default Normal.
- **kvcache_dtype - string**:
    - Indicates the KV cache data type. Valid values: [Float16, Int4_to_F8, Int4_to_F16]. Float16 uses float16 storage; Int4_to_F8 uses 4-bit integer quantization storage and converts to 8-bit float during computation; Int4_to_F16 uses 4-bit integer quantization storage and converts to 16-bit float during computation. Default Float16.

- **kvcache_buffer_len - int**:
  - Maximum length of KV cache, in tokens; must be less than the model's max_position_embeddings.
- **kvcache_group_size - int**:
  - Effective when kvcache_store_method is GroupQuant, indicating the group size for group quantization; when kvcache_store_method is Normal, defaults to 32.
- **kvcache_residual_depth - int**:
  - Effective when kvcache_store_method is GroupQuant, indicating that group quantization is triggered after storing this number of tokens; when kvcache_store_method is Normal, defaults to 32.
- **sliding_window_size - int**:
  - Effective when attention_type is SlidingAttention, indicating the sliding window size; when attention_type is FullAttention, defaults to -1.

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`):

---

exPagedAttention

An attention subgraph operator using a paged KV cache, supporting FullAttention, SlidingAttention, and multi-request Decode.

FORMULA

```
Q = rope(query, cos_pos_emb, sin_pos_emb) if rope == 1 else query
K_current = rope(key, cos_pos_emb, sin_pos_emb) if rope == 1 else key
K_all, V_all = concat(valid historical KV cache, valid K_current/value from the current iteration)
logits[h,q,k] = scale * dot(Q[h,q], K_all[kv_head(h),k]) + mask[q,k]
P[h,q,k] = exp(logits[h,q,k]) / (sum_j exp(logits[h,q,j]) + (exp(sinks[h]) if has_sink == 1 else 0))
output[h,q] = sum_k P[h,q,k] * V_all[kv_head(h),k]
```

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **query** - `T1` (required):
  - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]
  - num_attention_heads must be an integer multiple of num_key_value_heads. MHA, MQA, and GQA are supported.

- **key** - `T1` (required):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **value** - `T1` (required):
  - Shape: NCHW[num_key_value_heads, value_head_dim, 1, seq_len]
- **attention_mask** - `T1` (required):
  - Shape: NCHW[1, seq_len, 1, seq_len]
  - An additive mask between tokens in the current iteration. The valid historical-cache range and the SlidingAttention window mask are handled by the internal subgraph.
- **sinks** - `T3` (optional):
  - Shape: N[num_attention_heads]
  - When has_sink=1, this input immediately follows attention_mask. It must be an FP32 initializer and must also be declared as a graph input, as required by the current importer. There is one logit per query head, added only to the Softmax denominator. It is excluded from scale, RoPE, and mask processing and is not written to the KV cache. Subsequent inputs shift accordingly.
- **cos_pos_emb** - `T1` (optional):
  - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
  - Required when rope=1; placed after attention_mask or the optional sinks input.
- **sin_pos_emb** - `T1` (optional):
  - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
  - Required when rope=1; immediately follows cos_pos_emb.
- **control_variables** - `T2` (required):
  - Shape: NC[request_num, 9]
  - One row per request, with nine columns in this exact order: [Th, Tc, Ts, Tsr, longest_surviving_id, store_begin_id, Tr, Ti, request_masks].
    Th is the historical token count before the current iteration. Tc is the valid token count in the current iteration, at most bucket_size and equal to 1 during Decode.
    Ts is the aligned system-prompt length; it must be a multiple of 32 and less than kvcache_buffer_len. Tsr is the actual system-prompt length, satisfying 0 <= Tsr <= Ts. longest_surviving_id is the storage position of the oldest still-valid token in the main cache, or -1 for an empty cache. store_begin_id is the next write position in the main cache.
    Tr is the valid token count in the residual cache. Ti is the logical token position at the end of the previous iteration; it is compared with Th to handle rollback.
    In multi-request mode, request_masks controls Attention and KVStore for this request: 0 skips execution, and nonzero executes it. In single-request mode, it is treated as 1. longest_surviving_id, store_begin_id, Tr, and Ti are mutable state and must be carried forward with the corresponding KV cache for subsequent inference. Th/Tc/Ts/Tsr are supplied by the caller.
    A new request can be initialized with [0, Tc, Ts, Tsr, -1, 0, 0, 0, 1]. Cache state is reset internally when Th=0.

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[num_attention_heads, value_head_dim, 1, seq_len]

ATTRIBUTES

- **attention_type - string**:
  - Explicitly selects full-attention or sliding-window-attention mode.
- **scale - float**:
  - The scale factor for QK logits, defaulting to 1.0. For standard scaled dot-product attention, the caller must set it to 1/sqrt(head_dim).
- **has_sink - int**:
  - Defaults to 0. A value of 1 indicates a constant sinks input; supported in both FullAttention and SlidingAttention.
- **rope - int**:
  - Defaults to 0. A value of 1 applies RoPE internally to query/key and requires both cos_pos_emb and sin_pos_emb. Set to 0 when RoPE is applied externally.
- **kvcache_store_method - string**:
  - Specifies the KV-cache data type. Float16 stores float16 values. Int4_to_F8 uses 4-bit integer quantized storage and converts to 8-bit floating point for computation. Int4_to_F16 uses 4-bit integer quantized storage and converts to 16-bit floating point for computation. Defaults to Float16.
- **kvcache_dtype - string**:
  - Normal uses Float16. Other types are used for GroupQuant; the F8/F16 suffix indicates the precision used by the computation path. If unspecified, the compiler configuration is used.
- **kvcache_buffer_len - int**:
  - The maximum token capacity of the main KV cache per request; must be positive. If unspecified or set to -1, the compiler configuration is used.
- **tokens_per_page - int**:
  - The number of KV-cache tokens per page, defaulting to 512; must be positive and is capped at kvcache_buffer_len. FullAttention uses this page size; SlidingAttention uses a single page covering the entire kvcache_buffer_len.
- **kvcache_group_size - int**:
  - Effective when kvcache_store_method is GroupQuant, indicating the group size for group quantization; when kvcache_store_method is Normal, defaults to 32.
- **kvcache_residual_depth - int**:
  - Effective when kvcache_store_method is GroupQuant, indicating that group quantization is triggered after storing this number of tokens; when

kvcache_store_method is Normal, defaults to 32.

- **request_num - int**:
  - Defaults to 1. The actual request count is determined statically by dimension 0 of control_variables; this attribute must match that dimension.
- **sliding_window_size - int**:
  - The window size for SlidingAttention; ignored for FullAttention. Defaults to -1.

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`): **T3** in (`tensor<float>`):

---

exSDPAttention

Performs scaled dot-product attention with optional mask.

FORMULA

```
q = Query
k = Key
v = Value

score = MatMul(k, q) * scale
if Mask is not None:
    score = score + Mask

prob = Softmax(score, axis=1)
Y = MatMul(v, prob)
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::scaled_dot_product_attention`

INPUTS

- **Query** - `T1` (required):
  - Shape: NCHW[batch, query_length, head, query_channel]
  - Quantization: per-layer
  - q matrix input, corresponding to query in the code.
- **Key** - `T1` (required):
  - Shape: NCHW[batch, key_length, head, key_channel]
  - Quantization: per-layer

    - k matrix input, corresponding to key in the code; participates in qk_matmul.
- **Value** - `T1` (required):
    - Shape: NCHW[batch, value_length, head, value_channel]
    - Quantization: per-layer
    - v matrix input.
- **Mask** - `T1` (optional):
    - Shape: NCHW[batch, key_channel, head, query_channel]
    - Quantization: per-layer
    - attention mask; when present, it is added to the qk_matmul result before softmax. The shape must be broadcastable to the qk_matmul output.

OUTPUTS

- **SDPAOutput** - `T1`:
    - Shape: NCHW[batch, query_length, head, value_channel]
    - Quantization: per-layer
    - Final output of scaled dot-product attention.

ATTRIBUTES

- **scale - float**:
    - Default value 1.0, applied to the qk_matmul result.
- **QMatmul - string**:
    - Quantization parameter description for qk_matmul output.
- **QAdd - string**:
    - Quantization parameter description for mask add output; only used when Mask is present.
- **QSoftmax - string**:
    - Quantization parameter description for softmax output.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

---

exLinearAttention

Chunk Gated Delta Rule linear attention subgraph operator; computes recursive linear attention of query/key/value in chunks.

FORMULA

`output = LinearAttention(query, key, value, g, beta, initial_state)`

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **query** - `T1` (required):
  - Shape: HD1S[H, D, 1, S]
  - Quantization: per-layer
  - H is num_heads, D is head_dim, S is seq_len.
- **key** - `T1` (required):
  - Shape: HD1S[H, D, 1, S]
  - Quantization: per-layer
- **value** - `T1` (required):
  - Shape: HD1S[H, D, 1, S]
  - Quantization: per-layer
- **g** - `T1` (required):
  - Shape: 1H1S[1, H, 1, S]
  - Quantization: per-layer
  - Gate factor.
- **beta** - `T1` (required):
  - Shape: 1H1S[1, H, 1, S]
  - Quantization: per-layer
- **Tc** - `T2` (required):
  - Shape: S[seq_len]
  - Effective seq_len scalar. When Tc < seq_len, left-side zero padding is applied to query/key/value along the seq dimension.
- **initial_state** - `T1` (optional):
  - Shape: HDD[H, D, 1, D]
  - Quantization: per-layer
  - Optional initial recurrent state. If provided by the frontend, it is directly connected.

OUTPUTS

- **output** - `T1` :
  - Shape: HD1S[H, D, 1, S]

    - Quantization: per-layer
- **final_state** - `T1` :
    - Shape: HDD[H, D, 1, D]
    - Quantization: per-layer
    - Only output when OutputFinalState is true.

ATTRIBUTES

- **ChunkSize - int**:
    - Chunk size, default 32. seq_len must be a multiple of ChunkSize (padding is the caller's responsibility).
- **UseQkL2Norm - bool**:
    - Whether to apply L2 normalization to query/key; default false.
- **OutputFinalState - bool**:
    - Whether to output the final recurrent state; default false.

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>` , `tensor<bfloat16>`): **T2** in (`tensor<int32>`):

NOTES

Subgraph operator; the actual implementation is expanded by createSubGraph at the IR layer; emit only returns success.
The batch dimension is fixed to 1. seq_len must be padded to a multiple of ChunkSize before entering the subgraph.

---

exRoPE

General subgraph operator supporting RoPE positional encoding.

FORMULA

```
output = input * cos_pos_emb + rotate_half(input) * sin_pos_emb
```

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **input** - `T1` (required):
    - Shape: NCHW[batch_size, channel, 1, seq_len]
- **cos_pos_emb** - `T1` (required):

    - Shape: NCHW[batch_size, channel, 1, position_embeddings]
    - position_embeddings == 1 or
      seq_len <= position_embeddings <= max_position_embeddings。
- **sin_pos_emb** - `T1` (required):
    - Shape: NCHW[batch_size, channel, 1, position_embeddings]
    - position_embeddings == 1 or
      seq_len <= position_embeddings <= max_position_embeddings。
- **Th** - `T2` (optional):
    - Shape: N[1]
    - history token len; the number of tokens previously processed.
- **Tc** - `T2` (optional):
    - Shape: N[1]
    - current token len; the number of valid tokens in the current inference bucket.
- **Ts** - `T2` (optional):
    - Shape: N[1]
    - system prompt token len; the aligned system prompt length. Both [182X, 3572] require alignment to a multiple of 32.
- **Tsr** - `T2` (optional):
    - Shape: N[1]
    - Real system prompt token len; the actual system prompt length, less than or equal to Ts.

OUTPUTS

- **output** - `T1` :
    - Shape: NCHW[batch_size, channel, 1, seq_len]

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`):

NOTES

When exRoPE is used in an LLM model and
full cos/sin_pos_emb is provided (position embeddings > seq_len), inputs such as Th/Tc/Ts/Tsr will be present. exRoPE internally calculates the actual length of cos/sin_pos_emb to use based on Th/Tc/Ts/Tsr, thereby supporting dynamic sequence length inference. In other cases, seq_len == position_embeddings and
Th/Tc/Ts/Tsr do not exist.

---

exSum

Weighted sum of multiple inputs, commonly used for merging LoRA multi-branch results.

FORMULA

`Y = Sum(X_i * scale_i) for i in 0..N-1`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::sum`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Variable-length input list; at least 1, all input shapes must be identical.

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **scales - float[scale_1, ..., scale_N-1]**:
  - Weighting coefficient for each extra input (except the first); the count is the number of inputs - 1.

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`):

Type Conversion

Cast

Converts each element of the input tensor to the specified target data type.

FORMULA

`output = Cast(input, to)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::to`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **to - int**:
  - Target data type, mapped from ONNX to_type; supports bool/int8/uint8/int16/int32/float16/float32.

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`): **T2** in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`):

---

exDataConvert

Converts the data type of the input tensor to the specified target type.

FORMULA

`output = exDataConvert(input, input_type, output_type)`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::to`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **input_type - int**:
  - Input data type.
- **output_type - int**:
  - Output data type.

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`): **T2** in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`):

I/O Processing

exInputProc

Performs input data preprocessing.

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **B** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<float32>` , `tensor<float16>` , `tensor<bool>` , `tensor<uint8>`): **T2** in (`tensor<float16>` , `tensor<int8>`):

---

exOutputProc

Performs output data post-processing.

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **B** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>` , `tensor<int8>`): **T2** in (`tensor<float32>` , `tensor<float16>` , `tensor<bool>` , `tensor<uint8>`):

RK3572

Convolution

Conv

Convolution computation.

FORMULA

`output = Conv(input_tensor, weight, bias)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::_convolution`

INPUTS

- **input_tensor** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **weight** - `T1` (required):
  - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
  - Quantization: per-layer, per-channel
  - kernel_h range: [1, 31]
  - kernel_w range: [1, 31]
- **bias** - `T1` (optional):
  - Shape: C[output_channel]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **strides - int[stride_h, stride_w]**:

    - stride_h range: [1, 7]
    - stride_w range: [1, 7]
- **pads - int[pads_top, pads_left, pads_bottom, pads_right]**:
    - pads_top range: [0, 15]
    - pads_left range: [0, 15]
    - pads_bottom range: [0, 15]
    - pads_right range: [0, 15]
- **group - int**: —
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h range: [1, 31]
    - dilation_w range: [1, 31]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

SPECIAL CASES

Depthwise Convolution When Conv is used with depthwise convolution group semantics, the following differential specifications apply.

- **Input Differences — weight**: kernel_h range: [1, 8]; kernel_w range: [1, 8]
- **Attribute Differences — dilations**: dilation_h range: [1, 32]; dilation_w range: [1, 32]

ConvTranspose

Transposed convolution computation.

FORMULA

```
output = ConvTranspose(input_tensor, weight, bias)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::conv_transpose1d` , `aten::conv_transpose2d`

INPUTS

- **input_tensor** - `T1` (required):
    - Shape: NCHW[batch, channel, height, width]

    - Quantization: per-layer
    - When dilated_kernel_h > 1, input_tensor.width < 16383.
      The first layer input width must also satisfy the model input limitations.
- **weight** - `T1` (required):
    - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
    - Quantization: per-layer, per-channel
    - kernel_h range: [1, 31]
    - kernel_w range: [1, 31]
- **bias** - `T1` (optional):
    - Shape: C[output_channel]
    - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer

ATTRIBUTES

- **strides - int[stride_h, stride_w]**:
    - stride_h range: [1, 8]
    - stride_w range: [1, 8]
- **pads - int[pads_top, pads_left, pads_bottom, pads_right]**:
    - pads_top range: [0, 15]
    - pads_left range: [0, 15]
    - pads_bottom range: [0, 15]
    - pads_right range: [0, 15]
    - kernel_h * dilation_h - dilation_h - pads_top < 0 is not supported
      kernel_w * dilation_w - dilation_w - pads_left < 0 is not supported
      stride_h * (height - 1) - pads_top + 1 < output_h is not supported
      stride_w * (width - 1) - pads_left + 1 < output_w is not supported
- **group - int**:
    - Supports group = 1; when num_input = num_output, also supports group = num_output.
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h range: [1, 32]
    - dilation_w range: [1, 32]
    - When dilation != (1, 1), only kernel = (1, 1) is supported.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

ConvActivation

Fused Conv and generic Activation computation; activation type is determined by the Activation sub-operator.

FORMULA

```
output = Activation(Conv(input_tensor, weight, bias))
```

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

```
aten::_convolution
```

ConvRelu

Fused Conv and Relu computation.

FORMULA

```
output = Relu(Conv(input_tensor, weight, bias))
```

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

```
aten::_convolution
```

ConvClip

Fused Conv and Clip computation.

FORMULA

`output = Clip(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvSigmoid

Fused Conv and Sigmoid computation.

FORMULA

`output = Sigmoid(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvTanh

Fused Conv and Tanh computation.

FORMULA

`output = Tanh(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvElu

Fused Conv and Elu computation.

FORMULA

`output = Elu(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvHardSigmoid

Fused Conv and HardSigmoid computation.

FORMULA

`output = HardSigmoid(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvHardSwish

Fused Conv and HardSwish computation.

FORMULA

`output = HardSwish(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvSwish

Fused Conv and Swish computation.

FORMULA

`output = Swish(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvSoftplus

Fused Conv and Softplus computation.

FORMULA

`output = Softplus(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvMish

Fused Conv and Mish computation.

FORMULA

```
output = Mish(Conv(input_tensor, weight, bias))
```

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvAdd

Fused Conv and Add computation.

FORMULA

```
output = Add(Conv(input_tensor, weight, bias), residual)
```

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

EXTRA INPUTS

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

- Second input of Add.
  Supports broadcasting; broadcast constraints are the same as Add.

ConvAddRelu

Fused Conv, Add, and Relu computation.

FORMULA

`output = Relu(Add(Conv(input_tensor, weight, bias), residual))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

EXTRA INPUTS

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Second input of Add.
    Supports broadcasting; broadcast constraints are the same as Add.

ConvPRelu

Fused Conv and PRelu computation.

FORMULA

`output = PRelu(Conv(input_tensor, weight, bias))`

INHERITANCE

Inherits from Conv.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::_convolution`

ConvTransposeActivation

Fused ConvTranspose and generic Activation computation; activation type is determined by the Activation sub-operator.

FORMULA

```
output = Activation(ConvTranspose(input_tensor, weight, bias))
```

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeRelu

Fused ConvTranspose and Relu computation.

FORMULA

```
output = Relu(ConvTranspose(input_tensor, weight, bias))
```

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeClip

Fused ConvTranspose and Clip computation.

FORMULA

`output = Clip(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeSigmoid

Fused ConvTranspose and Sigmoid computation.

FORMULA

`output = Sigmoid(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeTanh

Fused ConvTranspose and Tanh computation.

FORMULA

`output = Tanh(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeElu

Fused ConvTranspose and Elu computation.

FORMULA

`output = Elu(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeHardSigmoid

Fused ConvTranspose and HardSigmoid computation.

FORMULA

`output = HardSigmoid(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeHardSwish

Fused ConvTranspose and HardSwish computation.

FORMULA

`output = HardSwish(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeSwish

Fused ConvTranspose and Swish computation.

FORMULA

`output = Swish(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeSoftplus

Fused ConvTranspose and Softplus computation.

FORMULA

`output = Softplus(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeMish

Fused ConvTranspose and Mish computation.

FORMULA

`output = Mish(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

ConvTransposeAdd

Fused ConvTranspose and Add computation.

FORMULA

`output = Add(ConvTranspose(input_tensor, weight, bias), residual)`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

EXTRA INPUTS

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

- Second input of Add.
  Supports broadcasting; broadcast constraints are the same as Add.

ConvTransposePRelu

Fused ConvTranspose and PRelu computation.

FORMULA

`output = PRelu(ConvTranspose(input_tensor, weight, bias))`

INHERITANCE

Inherits from ConvTranspose.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::conv_transpose2d`

---

exConvStreaming

Convolution computation with streaming cache.

FORMULA

`y = Conv(Concat(cache, x), w, b)`
`cache = Slice(Concat(cache, Slice(x, width=[0, tc])), width=[end - cache_w, end])`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::_convolution`

INPUTS

- **x** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **w** - `T1` (required):

    - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
    - Quantization: per-layer, per-channel
    - kernel_h range: [1, 31]
    - kernel_w range: [1, 31]
- **b** - `T1` (optional):
    - Shape: C[output_channel]
    - Quantization: per-layer
    - Optional; when there is no bias but tc needs to be passed, b can be an empty input placeholder.
- **tc** - `T2` (optional):
    - Shape: N[1]
    - tc indicates the effective width written to cache this time; the length must be less than the input width.
      When there are 3 inputs, if the size of the third input equals the output_channel of weight, it is treated as b; otherwise as tc.

OUTPUTS

- **y** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer

ATTRIBUTES

- **strides - int[stride_h, stride_w]**:
    - stride_h range: [1, 7]
    - stride_w range: [1, 7]
- **pads - int[pads_top, cache_w, pads_bottom, pads_right]**:
    - pads_top range: [0, 15]
    - pads_bottom range: [0, 15]
    - pads_right range: [0, 15]
    - cache_w is the streaming cache length; must be greater than 0.
- **group - int**: —
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h range: [1, 31]
    - dilation_w range: [1, 31]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

SPECIAL CASES

Depthwise exConvStreaming When exConvStreaming is used with depthwise convolution group semantics, the following differential specifications apply.

- **Input Differences** — **w**: kernel_h range: [1, 8]; kernel_w range: [1, 8]
- **Attribute Differences** — **dilations**: dilation_h range: [1, 32]; dilation_w range: [1, 32]

NOTES

Subgraph operator; the actual implementation is expanded by createSubGraph at the IR layer; emit only returns success.

Pooling

AveragePool

Performs average pooling on the input tensor with the specified kernel_shape, strides, and pads.

FORMULA

`Y = average_pool(X, kernel_shape, strides, pads)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::adaptive_avg_pool2d`, `aten::avg_pool2d`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - int8/int16 quantization requires height <= 32 and width <= 32

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **kernel_shape - int[kernel_h, kernel_w]**:
  - kernel_h range: [1, 32]
  - kernel_w range: [1, 32]
- **strides - int[stride_h, stride_w]**:
  - stride_h range: [1, 63]
  - stride_w range: [1, 63]
- **pads - int[pad_top, pad_left, pad_bottom, pad_right]**:
  - pad_top range: [0, 63]
  - pad_left range: [0, 63]
  - pad_bottom range: [0, 63]
  - pad_right range: [0, 63]
- **auto_pad - string**:
  - Only explicit pads are supported.
- **ceil_mode - int**:
  - Ceil mode is supported by adding bottom/right padding.
- **count_include_pad - int**:
  - When set to 1, the mean calculation includes the padding area.
- **dilations - int[dilation_h, dilation_w]**:
  - dilation_h range: [1, 1]
  - dilation_w range: [1, 1]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

MaxPool

Performs max pooling on the input tensor with the specified kernel_shape, strides, and pads.

FORMULA

`Y = max_pool(X, kernel_shape, strides, pads)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::max_pool1d`, `aten::max_pool2d`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **kernel_shape - int[kernel_h, kernel_w]**:
  - kernel_h range: [1, 32]
  - kernel_w range: [1, 256]
  - kernel_h * kernel_w <= 256
- **strides - int[stride_h, stride_w]**:
  - stride_h range: [1, 31]
  - stride_w range: [1, 256]
- **pads - int[pad_top, pad_left, pad_bottom, pad_right]**:
  - pad_top range: [0, 63]
  - pad_left range: [0, 63]
  - pad_bottom range: [0, 63]
  - pad_right range: [0, 63]
- **auto_pad - string**:
  - Only explicit pads are supported.
- **ceil_mode - int**:
  - Ceil mode is supported by adding bottom/right padding.
- **dilations - int[dilation_h, dilation_w]**:
  - dilation_h range: [1, 1]
  - dilation_w range: [1, 1]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int64>`):

GlobalAveragePool

Performs global average pooling on the H/W spatial dimensions of the input tensor.

FORMULA

```
Y = mean(X, axes=[2, 3])
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

```
aten::adaptive_avg_pool2d
```

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - int8 quantization requires height <= 32 and width <= 32;

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, 1, 1]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

GlobalMaxPool

Performs global max pooling on the spatial dimensions of the input tensor, outputting the maximum value per channel.

FORMULA

```
Y[batch, channel, 0, 0] = max(X[batch, channel, :, :])
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::adaptive_max_pool2d`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Hard constraint: height * width <= 256 and height <= 32.

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, 1, 1]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Elementwise Operations

Add

Performs element-wise binary addition, supporting NumPy-style broadcasting.

FORMULA

`C = A + B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::add` , `aten::add_`

INPUTS

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]

    - Quantization: per-layer
    - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1, T2`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer
    - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<int32>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Sub

Performs element-wise binary subtraction, supporting NumPy-style broadcasting.

FORMULA

`C = A - B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sub` , `aten::sub_`

INPUTS

- **A** - `T1, T2`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer
    - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1, T2`:

- Shape: NCHW[batch, channel, height, width]
- Quantization: per-layer
- Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Mul

Performs element-wise binary multiplication, supporting NumPy-style broadcasting.

FORMULA

`C = A * B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::mul` , `aten::mul_`

INPUTS

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Div

Performs element-wise binary division, supporting NumPy-style broadcasting.

FORMULA

`C = A / B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::div` , `aten::true_divide`

INPUTS

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`): **T2** in (`const_tensor<float16>`):

Min

Computes the element-wise minimum of multiple input tensors, supporting NumPy-style broadcasting.

FORMULA

`C = min(A, B, ...)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::min` , `aten::minimum`

INPUTS

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer, per-channel

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Max

Computes the element-wise maximum of multiple input tensors, supporting NumPy-style broadcasting.

FORMULA

`C = max(A, B, ...)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::max`, `aten::maximum`

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer, per-channel

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Pow

Performs element-wise exponentiation.

FORMULA

`output = input ^ exponent`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::pow`, `aten::pow_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **exponent** - `T2`:
  - Shape: S[scalar]

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Neg

Performs element-wise negation on the input tensor.

FORMULA

`Y = -X`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::neg`, `aten::neg_`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Exp

Computes the element-wise base-e exponential of the input tensor.

FORMULA

```
output = exp(input)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::exp` , `aten::exp_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Sqrt

Performs element-wise square root on the input tensor.

FORMULA

```
output = sqrt(input)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sqrt` , `aten::sqrt_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Reciprocal

Performs element-wise reciprocal on the input tensor.

FORMULA

```
Y = 1 / X
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::reciprocal`, `aten::reciprocal_`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Floor

Performs element-wise floor (rounds towards negative infinity) on the input tensor.

FORMULA

```
output = floor(input)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::floor`, `aten::floor_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Round

Rounding function using round-half-up semantics, formula: y = Floor(x + 0.5). Differs from ONNX Round's round-half-to-even (banker's rounding).

FORMULA

```
y = Floor(x + 0.5)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::round` , `aten::round_`

INPUTS

- **x** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`):

Sin

Computes the element-wise sine function.

FORMULA

```
y = sin(x)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sin`, `aten::sin_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

NOTES

Implementation first applies mod 2π to the input, then performs a LUT lookup.

Cos

Computes the element-wise cosine function.

FORMULA

`y = cos(x)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::cos`, `aten::cos_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

NOTES

Implementation first applies mod 2π to the input, then performs a LUT lookup.

Atan

Computes the elementwise arctangent.

FORMULA

```
y = atan(x)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::atan` , `aten::atan_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

NOTES

The main LUT covers x in [-16, 16]. Outside this range, interpolation uses the slope 1/(1+x^2), and the output converges to +/-pi/2.

Tan

Computes the elementwise tangent.

FORMULA

`y = tan(x)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::tan`, `aten::tan_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`, `tensor<bfloat16>`):

NOTES

The implementation first applies mod pi to the input (result in [-pi/2, pi/2]), then looks up the LUT. The main LUT only covers [-1, 1]; values outside this range are clamped to tan(+/-1). Accuracy is therefore not guaranteed for inputs whose mod-pi result lies outside [-1, 1].

---

exFastExp

Computes the element-wise natural exponential using an approximation for improved performance.

FORMULA

`output ≈ exp(input)`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::exp`, `aten::exp_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

---

exTwoPow

Computes the element-wise power of 2.

FORMULA

`y = 2^x`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::pow`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

AddRelu

Fused Add and Relu computation.

FORMULA

```
C = Relu(A + B)
```

INHERITANCE

Inherits from Add.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::add` , `aten::add_`

MulRelu

Fused Mul and Relu computation.

FORMULA

```
C = Relu(A * B)
```

INHERITANCE

Inherits from Mul.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

`aten::mul`, `aten::mul_`

Activation Functions

Relu

Rectified Linear Unit, performs max(input, 0) element-wise.

FORMULA

```
output = max(input, 0)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::relu`, `aten::relu6`, `aten::relu_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Sigmoid

Applies the Sigmoid function element-wise to the given input tensor.

FORMULA

```
y = 1 / (1 + exp(-x))
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sigmoid`, `aten::sigmoid_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Clip

Clips the input tensor elements to the [min, max] range; out-of-bounds values are clamped to the boundary.

FORMULA

```
output = Clip(input, min, max)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::clamp`, `aten::clamp_`

INPUTS

- **input** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **min** - `T2` (optional):

- Shape: S[scalar]
- Clip lower bound, scalar. Omitted means no lower bound.

- **max** - `T2` (optional):
  - Shape: S[scalar]
  - Clip upper bound, scalar. Omitted means no upper bound.

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

LeakyRelu

Leaky Rectified Linear Unit; positive values remain unchanged, negative values are scaled by a fixed ratio alpha.

FORMULA

```
output = input if input >= 0 else alpha * input
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::leaky_relu` , `aten::leaky_relu_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **alpha - float[alpha]**:
  - alpha range: [0, 1]
  - Negative value scaling factor.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

PRelu

Parametric Rectified Linear Unit; positive values remain unchanged, negative values are scaled by a learnable parameter slope.

FORMULA

```
output = input if input >= 0 else slope * input
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::prelu` , `aten::prelu_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer, per-channel
- **slope** - `T2` :
  - Shape: C[channel]
  - Quantization: per-channel
  - Supports a single scalar or C-dimension coefficients.

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer, per-channel

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Elu

Applies the Elu function element-wise, alpha = 1.

FORMULA

`f(x) = alpha * (exp(x) - 1) for x < 0; f(x) = x for x >= 0`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::elu` , `aten::elu_`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Softplus

Applies the Softplus function element-wise.

FORMULA

`y = ln(exp(x) + 1)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::softplus`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

HardSigmoid

Applies the HardSigmoid function element-wise, controlled by alpha and beta attributes.

FORMULA

```
y = max(0, min(1, alpha * x + beta))
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::hardsigmoid` , `aten::hardsigmoid_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

ATTRIBUTES

- **alpha - float**:
  - Default 0.2 (consistent with the ONNX standard).
- **beta - float**:
  - Default 0.5.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

HardSwish

Applies the HardSwish function element-wise, alpha = 1/6, beta = 0.5.

FORMULA

```
y = x * max(0, min(1, alpha * x + beta))
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::hardswish` , `aten::hardswish_`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Tanh

Computes the element-wise hyperbolic tangent of the given input tensor.

FORMULA

`y = tanh(x)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::tanh` , `aten::tanh_`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Mish

Applies the Mish function element-wise.

FORMULA

`y = x * tanh(softplus(x)) = x * tanh(ln(1 + exp(x)))`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::mish`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exGelu

Applies the Gelu function element-wise.

FORMULA

`y = x * Phi(x), when Phi uses tanh approximation: y = 0.5 * x * (1 + tanh(sqrt(2/pi) * (x + 0.044715 * x^3)))`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::gelu`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exHardSwish

Applies the HardSwish function element-wise, alpha = 1/6, beta = 0.5.

FORMULA

```
y = x * max(0, min(1, alpha * x + beta))
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::hardswish` , `aten::hardswish_`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exMish

Applies the Mish function element-wise.

FORMULA

```
y = x * tanh(softplus(x)) = x * tanh(ln(1 + exp(x)))
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::mish`, `aten::mish_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

---

exSwish

Applies the Swish function (also known as SiLU) element-wise.

FORMULA

```
y = x * sigmoid(x)
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::silu`, `aten::silu_`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exGlu

Splits the input tensor into two equal parts along the specified axis, applies sigmoid gating to the second part, and multiplies it with the first part.

FORMULA

```
a, b = np.split(X, 2, axis=axis)
Y = a * sigmoid(b)
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::glu`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - The length of the dimension specified by axis must be even.
    The NPU subgraph implementation primarily targets axis = 1 channel dimension splitting.

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, out_channel, height, width]
  - Quantization: per-layer
  - The output shape is consistent with the input shape, but the dimension specified by axis becomes half of the input.
    When axis = 1, out_channel = channel / 2.

ATTRIBUTES

- **axis - int**:

- Specifies the dimension for GLU split; valid range is [-rank, rank - 1].
  Negative axes are converted to positive axes following NumPy semantics.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exLogit

Applies the logit activation function to the input tensor.

FORMULA

```
Y = log(X / (1 - X))
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::logit`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Comparison & Logical Operations

Equal

Compares whether two tensors are equal element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

`C = (A == B)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::eq` , `aten::eq_`

INPUTS

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **B** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **C** - `T2` :
  - Shape: NCHW[batch, channel, height, width]
  - Boolean semantic result; RKNN stores it in int8 memory layout.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Greater

Compares whether A is greater than B element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

`C = (A > B)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::gt` , `aten::gt_`

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.

OUTPUTS

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

GreaterOrEqual

Compares whether A is greater than or equal to B element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

```
C = (A >= B)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::ge` , `aten::ge_`

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- Quantization: per-layer

OUTPUTS

- **C** - `T2` :
  - Shape: NCHW[batch, channel, height, width]
  - Boolean semantic result; RKNN stores it in int8 memory layout.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Less

Compares whether A is less than B element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

```
C = (A < B)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::lt` , `aten::lt_`

INPUTS

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.
- **B** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.

OUTPUTS

- **C** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

LessOrEqual

Compares whether A is less than or equal to B element-wise, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

```
C = (A <= B)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::le` , `aten::le_`

INPUTS

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.
- **B** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.

OUTPUTS

- **C** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Not

Performs element-wise logical NOT on the input tensor.

FORMULA

`Y = !X`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::logical_not`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>`):

And

Performs element-wise logical AND, outputs a bool result, supporting NumPy-style broadcasting.

FORMULA

`C = A && B`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::bitwise_and`, `aten::logical_and`

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>` , `tensor<int8>`):

Where

Selects elements from x or y based on the condition, supporting NumPy-style broadcasting.

FORMULA

```
output = select(condition, x, y)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::where`

INPUTS

- **condition** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
- **x** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.
- **y** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting.

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>`): **T2** in (`tensor<int8>` , `tensor<float16>`):

Reduction Operations

ReduceMax

Computes the maximum of the input tensor along the specified axis, supporting multiple axes and keepdims.

FORMULA

```
output = ReduceMax(input, axes, keepdims)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::amax` , `aten::max`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - When keepdims=1, the reduced axis has size 1; when keepdims=0, the axis is removed from the output. The actual output rank depends on axes and keepdims.

ATTRIBUTES

- **axes - int[]**:

- Axis for computing the maximum value.

- **keepdims - int**:
  - Whether to keep the reduced dimensions (1 keeps, 0 removes).

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

ReduceMean

Reduces the input tensor by computing the mean along the specified axes.

FORMULA

`Y = mean(data, axes)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::mean`

INPUTS

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - For int8/int16 quantization with axes = [2], [3], [2, 3], or
    [1, 2, 3], height <= 32 and
    width <= 32 are required
- **axes** - `T2` :
  - Shape: Axes[num_axes]
  - Specifies the dimensions to reduce; negative axes are converted to positive axes following NumPy semantics.
    Supports axes = [1], [2], [3], [2, 3], [1, 2, 3]

OUTPUTS

- **reduced** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`const_tensor<int64>`):

ReduceSum

Reduces the input tensor by computing the sum along the specified axes.

FORMULA

`Y = sum(data, axes)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::sum`

INPUTS

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - For int8/int16 quantization with axes = [2], [3], [2, 3], or [1, 2, 3], height <= 32 and width <= 32 are required
- **axes** - `T2` :
  - Shape: Axes[num_axes]
  - Specifies the dimensions to reduce; negative axes are converted to positive axes following NumPy semantics.
    Supports axes = [1], [2], [3], [2, 3], [1, 2, 3]

OUTPUTS

- **reduced** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`const_tensor<int64>`):

ArgMax

Returns the indices of the maximum values along the specified axis.

FORMULA

`output = ArgMax(input, axis)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::argmax`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **axis - int**:
  - Specifies the axis for argmax operation; valid range [0, ndim-1].
- **keepdims - int**:
  - Whether to keep the output dimensions the same as the input. 0 means no, 1 means yes.
- **select_last_index - int**:
  - When multiple maximum values exist, whether to select the last index. 0 selects the first, 1 selects the last.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

ArgMin

Returns the indices of the minimum values along the specified axis.

FORMULA

`output = ArgMin(input, axis)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::argmin`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **axis - int**:
  - Specifies the axis for argmin operation; valid range [0, ndim-1].
- **keepdims - int**:
  - Whether to keep the output dimensions the same as the input. 0 means no, 1 means yes.
- **select_last_index - int**:
  - When multiple minimum values exist, whether to select the last index. 0 selects the first, 1 selects the last.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

TopK

Retrieves the K largest (or smallest, determined by largest) values and their indices along the specified axis.

FORMULA

`(values, indices) = TopK(X, K, axis, largest, sorted)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::topk`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **K** - `T3` :
  - Shape: S[scalar]
  - The top K values, scalar, stored in an int64 tensor.

OUTPUTS

- **values** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - The top K values.
- **indices** - `T2` :
  - Shape: NCHW[batch, channel, height, width]
  - Index of the corresponding value. The ONNX standard is int64; RKNN actually outputs int32 or int64.

ATTRIBUTES

- **axis - int**:
  - Axis for computation; default -1.
- **largest - int**:
  - 1 selects maximum, 0 selects minimum, default 1.
- **sorted - int**:
  - 1 outputs in descending order, 0 does not guarantee order, default 1.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>` , `tensor<int64>`): **T3** in (`tensor<int64>`):

Tensor Transform

Reshape

Returns a tensor with the specified shape without changing the input data or element count.

FORMULA

`output = reshape(input, shape)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::reshape`, `aten::reshape_`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **shape - int**:
  - Specifies the output tensor shape.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`):

Transpose

Transposes the input tensor according to the axis order specified by perm.

FORMULA

`Y = transpose(X, perm)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::permute`, `aten::transpose`, `aten::transpose_`

INPUTS

- **X** - `T1, T2`:
  - Shape: NCHW[n1, c1, h1, w1]
  - Quantization: per-layer
  - For detailed constraint rules, see the perm attribute list.

OUTPUTS

- **Y** - `T1, T2`:
  - Shape: NCHW[n2, c2, h2, w2]
  - Quantization: per-layer
  - For detailed constraint rules, see the perm attribute list.

ATTRIBUTES

- **perm - int[d0, d1, d2, d3]**:
  - Axis order for transpose. Constraints for different perm parameters are listed in the perm constraint table below.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`): **T2** in (`tensor<int16>`, `tensor<float16>`):

NOTES

Different perm parameter constraints are as follows (where p = 268435456, q = 65536):

perm	T1 Constraint	T2 Constraint
[0,1,2,3]	No restriction	No restriction
[0,1,3,2]	h1%16=0, h1*w1*<*16*p, h1*w1*<*2*q	h1%8=0, h1*w1*<*8*p, h1*w1*<*2*q
[0,2,3,1]	c1<8192, h1*w1%8=0, w1*c1<p*16	c1<8192, h1*w1%8=0, w1*c1<p*8
[0,2,1,3]	h1%16=0, h1*w1*<*16*p, h1*w1*<*2*q, c1*w1<p	h1%8=0, h1*w1*<*8*p, h1*w1*<*2*q, c1*w1<p
[0,3,1,2]	h1*w1*<*p, w1*<*8192, h1*c1<p	h1*w1*<*p, w1*<*8192, h1*c1<p
[0,3,2,1]	h1%16=0, h1*w1*<*16*p, h1*w1*<*2*q, c1<p*16	h1%8=0, h1*w1*<*8*p, h1*w1*<*2*q, c1<p*8
[1,0,2,3]	h1*w1*<*16*p, n1<8192, h1*w1<p	h1*w1*<*8*p, n1<8192, h1*w1<p

perm	T1 Constraint	T2 Constraint
[1,0,3,2]	h1%16=0, c1*h1*w1<16*p, h1*w1<2*q	h1%8=0, c1*h1*w1<8*p, h1*w1<2*q
[1,2,0,3]	n1*w1*<*p*16, c1*h1*w1<p, w1%16=0	n1*w1*<*p*8, c1*h1*w1<p, w1%8=0
[1,2,3,0]	h1*w1*<*p, n1*<*p*16, c1*h1*w1<p*16, n1*w1<p*16	h1*w1*<*p, n1*<*p*8, c1*h1*w1<p*8, n1*w1<p*8
[1,3,0,2]	c1*w1*<*8192, n1*h1<p, w1%16=0	c1*w1*<*8192, n1*h1<p, w1%8=0
[1,3,2,0]	c1*h1*w1<p, n1*h1<p, w1<8192, w1%16=0	c1*h1*w1<p, n1*h1<p, w1<8192, w1%8=0
[2,1,0,3]	c1<8192, c1*h1*w1<p, n1*w1<p	c1<8192, c1*h1*w1<p, n1*w1<p
[2,1,3,0]	c1*h1*w1<p, n1*w1*<*p*16, c1<8192, n1*w1%4=0	c1*h1*w1<p, n1*w1*<*p*8, c1<8192, n1*w1%4=0
[2,0,3,1]	c1<8192, c1*h1*w1<p	c1<8192, c1*h1*w1<p
[2,0,1,3]	n1*c1*h1*w1*<*p, h1*w1%16=0	n1*c1*h1*w1*<*p, h1*w1%8=0
[2,3,1,0]	c1*h1*w1<p, n1*c1*<*p, h1*w1<8192, h1*w1%16=0	c1*h1*w1<p, n1*c1*<*p, h1*w1<8192, h1*w1%8=0
[2,3,0,1]	h1*w1*<*8192, n1*c1<p	h1*w1*<*8192, n1*c1<p
[3,0,2,1]	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*w1%16=0	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*w1%8=0
[3,0,1,2]	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*h1*w1%16=0	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*h1*w1%8=0
[3,2,0,1]	h1*w1*<*p, n1*c1<8192, n1*c1*w1<p	h1*w1*<*p, n1*c1<8192, n1*c1*w1<p
[3,2,1,0]	c1*h1*w1<p, h1*w1*<*8192, 2*h1*w1*<*q, n1*c1<p*16, h1%16=0	c1*h1*w1<p, h1*w1*<*8192, 2*h1*w1*<*q, n1*c1<p*8, h1%8=0
[3,1,0,2]	h1*w1*<*p, c1*w1<8192, n1*h1<p	h1*w1*<*p, c1*w1<8192, n1*h1<p
[3,1,2,0]	h1*w1*<*p, n1*c1<8192, n1*h1*<*p, n1*c1*w1<p	h1*w1*<*p, n1*c1<8192, n1*h1*<*p, n1*c1*w1<p

---

Concat

Concatenates multiple tensors along the specified axis into a single tensor; all inputs must have the same size except along the axis dimension.

FORMULA

```
output = Concat(input_tensor_list, axis)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

```
aten::cat
```

INPUTS

- **input_tensor** - `T`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Input is a list of tensors; all tensors must have the same size except along the axis dimension.

OUTPUTS

- **output** - `T`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **axis - int[axis]**:
  - axis range: [-4, 3]
  - Concatenation axis; RKNN requires 4D NCHW input, supports negative axes.

DATA TYPE CONSTRAINTS

T in (`tensor<int8>` , `tensor<float16>`):

Split

Splits the input tensor into multiple output tensors along the specified axis.

FORMULA

```
outputs = Split(input_tensor, axis, num_outputs, split)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::split`

INPUTS

- **input_tensor** - `T`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - One or more output tensors; the count is determined by num_outputs, and the shape depends on the split.

ATTRIBUTES

- **axis - int[axis]**:
  - axis range: [-4, 3]
  - Split axis; RKNN requires 4D NCHW input, supports negative axes.
- **num_outputs - int**: —
- **split - int[]**:
  - Specifies the size of each output along the axis dimension after splitting.

DATA TYPE CONSTRAINTS

T in (`tensor<int8>` , `tensor<float16>`):

Expand

Broadcasts the input tensor to the target shape.

FORMULA

```
output = Expand(input, target_shape)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::expand` , `aten::expand_as`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Slice

Obtains a slice of the input tensor following NumPy slicing rules.

FORMULA

```
output = Slice(data, starts, ends, axes, steps)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::narrow` , `aten::slice`

INPUTS

- **data** - `T` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **starts** - `T2` (required):
  - Shape: D[dim]
  - Per-axis split start positions.
- **ends** - `T2` (required):
  - Shape: D[dim]
  - Per-axis split end positions.
- **axes** - `T2` (optional):
  - Shape: D[dim]
  - Selects the split axis; supports axes 0~3, supports multiple axes simultaneously; when omitted, all axes are processed.
- **steps** - `T2` (optional):
  - Shape: D[dim]

- Per-axis split step sizes. When height*width==1 and
  channel%subc==0 and
  starts[1]%subc==0 and
  ends[1]%subc==0 and
  steps[1]<=subc, steps can be non-1; otherwise, the NPU can only run when steps==[1,1,1,1].

OUTPUTS

- **output** - `T`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int64>`):

Pad

Pads the input tensor with the specified pads and mode.

FORMULA

```
output = Pad(data, pads, mode, constant_value)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::constant_pad_nd`

INPUTS

- **data** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **pads** - `T2` (required):
  - Shape: PADS[batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end, height_end, width_end]
- **constant_value** - `T1` (optional):
  - Shape: S[scalar]
  - Quantization: per-layer
  - Same type as data; only effective in constant mode, defaults to 0.

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **mode - string**:
  - constant mode has no range restrictions; edge mode only supports h/w directions.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`tensor<int64>`):

SPECIAL CASES

Reflect Pad In reflect mode, input shape components have additional range restrictions.

- **Input Differences** — **data**: channel range: [1, 8192]; height range: [1, 8192]; width range: [1, 8176]

DepthToSpace

Rearranges the channel dimension of the input tensor back to H/W spatial dimensions by blocksize.

FORMULA

```
N, C, H, W = X.shape
b = blocksize
out_channel = C // (b * b)
Y = np.empty((N, out_channel, H * b, W * b), dtype=X.dtype)

for n in range(N):
    for c in range(out_channel):
        for h in range(H * b):
            for w in range(W * b):
                offset = (h % b) * b + (w % b)
                in_c = c + offset * out_channel
                Y[n, c, h, w] = X[n, in_c, h // b, w // b]
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::pixel_shuffle`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - channel must be divisible by blocksize * blocksize.

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, out_channel, out_height, out_width]
  - Quantization: per-layer
  - out_channel = channel / (blocksize * blocksize)。
    out_height = height * blocksize。
    out_width = width * blocksize。

ATTRIBUTES

- **blocksize - int**:
  - Spatial block size; must be greater than 0.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>`):

SpaceToDepth

Rearranges H/W spatial blocks of the input tensor to the channel dimension.

FORMULA

```
N, C, H, W = X.shape
b = blocksize
Y = np.empty((N, C * b * b, H // b, W // b), dtype=X.dtype)

for n in range(N):
    for c in range(C):
        for h in range(H):
            for w in range(W):
                out_c = c + ((h % b) * b + (w % b)) * C
                Y[n, out_c, h // b, w // b] = X[n, c, h, w]
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::pixel_unshuffle`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - height and width must be divisible by blocksize.

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, out_channel, out_height, out_width]
  - Quantization: per-layer
  - out_channel = channel * blocksize * blocksize。
    out_height = height / blocksize。
    out_width = width / blocksize。

ATTRIBUTES

- **blocksize - int**:
  - Spatial block size; must be greater than 0.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>`):

Resize

Resizes the spatial dimensions of the input tensor according to the specified scales or sizes, supporting nearest and linear interpolation modes.

FORMULA

`Y = Resize(X, scales, mode, coordinate_transformation_mode, nearest_mode)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::upsample_bilinear2d`, `aten::upsample_nearest2d`

INPUTS

- **X** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **roi** - `T3` (optional):
  - Shape: N[8]
  - Crop region coordinates [start_x, start_y, start_z, start_t, end_x, end_y, end_z, end_t]; only the default value [0,0,0,0,1,1,1,1] is supported.
- **scales** - `T3` (optional):
  - Shape: N[4]
  - Per-dimension scale factors [batch_scale, channel_scale, height_scale, width_scale], mutually exclusive with sizes. In linear and
    nearest modes, only integer scaling is supported, and
    batch/channel dimension scale must be 1, H/W dimension scale <= 8.
- **sizes** - `T4` (optional):
  - Shape: N[4]
  - Target output size [batch, channel, height, width], mutually exclusive with scales. In linear and
    nearest modes, only integer scaling is supported, batch/channel dimensions remain unchanged, and
    H/W dimension multiplier <= 8.

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, out_height, out_width]
  - Quantization: per-layer

ATTRIBUTES

- **mode - string**:
  - linear and nearest modes are internally implemented via convolution simulation.
- **coordinate_transformation_mode - string**:
  - tf_crop_and_resize is not supported.
- **cubic_coeff_a - float**:
  - Cubic mode coefficient; currently only supports the default value -0.75.

- **exclude_outside - int**:
  - Excluding outer points is not supported.
- **extrapolation_value - float**:
  - Extrapolation value setting is not supported.
- **nearest_mode - string**:
  - Rounding method in nearest mode.
- **antialias - int**:
  - Antialiasing is not supported.
- **keep_aspect_ratio_policy - string**:
  - Only the default stretch mode is supported.
- **axes - int[axis_0, axis_1, axis_2, axis_3]**:
  - Specifies the scaling dimension; the current implementation does not impose additional constraints on this attribute.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T3** in (`tensor<float32>`): **T4** in (`tensor<int64>`):

---

exWindow

exWindow is derived from the window_partition and window_reverse operations in https://github.com/microsoft/Swin-Transformer/blob/main/models/swin_transformer.py . In the original paper, the input feature is divided into multiple non-overlapping windows, each of size window_size × window_size. Both input and output features must be 4D shapes, and division is performed along both height and width dimensions.

FORMULA

```
B, C, H, W = X.shape

if mode == "partition":
    x = X.reshape(B, C, num_windows[0], window_sizes[0],
                  num_windows[1], window_sizes[1])
    y = np.transpose(x, (0, 1, 2, 4, 3, 5))
    Y = y.reshape(B, C,
                  num_windows[0] * num_windows[1],
                  window_sizes[0] * window_sizes[1])

elif mode == "reverse":
    x = X.reshape(B, C, num_windows[0], num_windows[1],
                  window_sizes[0], window_sizes[1])
    y = np.transpose(x, (0, 1, 2, 4, 3, 5))
    Y = y.reshape(B, C,
                  num_windows[0] * window_sizes[0],
                  num_windows[1] * window_sizes[1])

elif mode == "partition_num_first":
    x = X.reshape(B, C, window_sizes[0], num_windows[0],
                  window_sizes[1], num_windows[1])
    y = np.transpose(x, (0, 1, 3, 5, 2, 4))
    Y = y.reshape(B, C,
                  num_windows[0] * num_windows[1],
                  window_sizes[0] * window_sizes[1])
```

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - partition requires height = num_h * win_h, width = num_w * win_w.
    reverse requires height = num_h * num_w, width = win_h * win_w.
    partition_num_first requires height = win_h * num_h, width = win_w * num_w.

OUTPUTS

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, out_height, out_width]
  - Quantization: per-layer
  - partition and partition_num_first output out_height = num_h * num_w, out_width = win_h * win_w.
    reverse outputs out_height = num_h * win_h, out_width = num_w * win_w.

ATTRIBUTES

- **mode - string**:
  - partition follows the window_partition implementation.
    reverse follows the window_reverse implementation.
    The difference between partition_num_first and partition is that partition_num_first divides H/W by num_windows, while partition divides H/W by window_sizes.
- **num_windows - int[num_h, num_w]**:
  - num_h range: [1, 8192]
  - num_w range: [1, 8192]
  - 2D vector, number of windows. Defines how many windows the height and width of the input feature are divided into.
- **window_sizes - int[win_h, win_w]**:
  - win_h range: [1, 8192]
  - win_w range: [1, 8192]
  - 2D vector, window size. Defines the height and width of each window. Attention computation and feature extraction within the window are based on this size.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Index & Scatter

Gather

Gathers elements from the input tensor along the specified axis based on indices.

FORMULA

```
output = Gather(input, indices, axis)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::embedding` , `aten::gather`

INPUTS

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

    - Quantization: per-layer
- **indices** - `T2` :
    - Shape: INDICES[batch, index]

OUTPUTS

- **output** - `T1` :
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer

ATTRIBUTES

- **axis - int**:
    - Gathers elements along this axis.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

GatherElements

Gathers elements from the input tensor along the specified axis by indices; the output shape matches that of indices.

FORMULA

```
output = GatherElements(data, indices, axis)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::gather`

INPUTS

- **data** - `T1` :
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer
- **indices** - `T2` :
    - Shape: NCHW[batch, channel, height, width]
    - Supports both constant and non-constant indices.

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **axis - int**:
  - Gathers elements along this axis; negative values are normalized to the valid range.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<int32>`): **T2** in (`tensor<int32>`):

GatherND

Gathers slices from the input tensor based on indices and combines them into a new output tensor.

FORMULA

```
output = GatherND(data, indices, batch_dims)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::gather_nd`

INPUTS

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **indices** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **batch_dims - int**:
  - Specifies how many leading dimensions are treated as batch dimensions; no gather operation is performed on these dimensions.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

ScatterND

Scatters updates into the data tensor at positions specified by indices to produce an output tensor, supporting reduction modes.

FORMULA

```
output = ScatterND(data, indices, updates, reduction)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::scatter_nd`

INPUTS

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **indices** - `T2` :
  - Shape: INDICES[num_index, index_dim]
- **updates** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **reduction - string**:
  - Specifies the reduction mode for scatter.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>` , `tensor<int64>`):

Normalization

BatchNormalization

Performs batch normalization on the input tensor (ref: https://arxiv.org/abs/1502.03167).

FORMULA

```
output = (input - mean) / sqrt(variance + epsilon)
```

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::batch_norm`

INPUTS

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
- **scale** - `T2` :
  - Shape: C[channel]
  - Quantization: per-channel
- **B** - `T2` :
  - Shape: C[channel]
  - Quantization: per-channel
- **mean** - `T2` :
  - Shape: C[channel]
  - Quantization: per-channel
  - During inference, it is folded into scale/B together with var and does not participate in computation at runtime.
- **var** - `T2` :

- Shape: C[channel]
- Quantization: per-channel
- During inference, it is folded into scale/B together with mean and does not participate in computation at runtime.

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **epsilon - float**:
  - A small real number added to the standard deviation to prevent division by zero; default 1e-5.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

BatchNormRelu

Fused BatchNormalization and Relu computation.

FORMULA

```
output = Relu(BatchNormalization(input))
```

INHERITANCE

Inherits from BatchNormalization.

OPERATOR LEVEL

Fused Operator

PYTORCH SOURCE

```
aten::batch_norm
```

---

exNorm

Performs extended normalization computation, with optional gamma scaling and beta shift.

FORMULA

```
x = X
weight = W if W is not None else 1
bias = B if B is not None else 0

if mode == "layer":
    x_centered = x - np.mean(x, axis=axes, keepdims=True)
    y = x_centered * fast_inv_sqrt(
        np.mean(x_centered * x_centered, axis=axes, keepdims=True) + eps)

elif mode == "mean_var":
    mean = np.mean(x, axis=axes, keepdims=True)
    var = np.mean(x * x, axis=axes, keepdims=True) - mean * mean
    y = (x - mean) * fast_inv_sqrt(var + eps)

elif mode == "rms":
    y = x * fast_inv_sqrt(
        np.mean(x * x, axis=axes, keepdims=True) + eps)

elif mode == "pixel":
    y = x * fast_inv_sqrt(
        np.sum(x * x, axis=axes, keepdims=True) + eps)

elif mode == "l1":
    y = x / np.maximum(
        np.sum(np.abs(x), axis=axes, keepdims=True), eps)

elif mode == "l2":
    y = x / np.maximum(
        fast_sqrt(np.sum(x * x, axis=axes, keepdims=True)), eps)

elif mode == "rms_reciprocal":
    y = fast_inv_sqrt(
        np.mean(x * x, axis=axes, keepdims=True) + eps)

elif mode == "gaussian_topk":
    mean = np.mean(x, axis=axes, keepdims=True)
    std = fast_sqrt(np.mean(x * x, axis=axes, keepdims=True) - mean * mean)
    cutoff_x = mean + std * std_multipler
    y = np.clip(x - cutoff_x, a_min=0, a_max=None).astype(x.dtype)

Y = y * weight + bias
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::layer_norm`, `aten::rms_norm`

INPUTS

- **X** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]

    - Quantization: per-layer
- **W** - `T1` (optional):
    - Shape: NCHW[batch, channel, height, width], [1]
    - Quantization: per-layer
    - Gamma scale parameter; supports per-layer, per-channel, or broadcast shapes compatible with the output.
- **B** - `T1` (optional):
    - Shape: NCHW[batch, channel, height, width], [1]
    - Quantization: per-layer
    - Beta shift parameter; supports per-layer, per-channel, or broadcast shapes compatible with the output.

OUTPUTS

- **Y** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - Quantization: per-layer
    - When mode is rms_reciprocal, the dimensions specified by axes maintain keepdims semantics and output as 1.

ATTRIBUTES

- **mode - string**:
    - Valid values: layer, mean_var, rms, pixel, l1, l2, rms_reciprocal, gaussian_topk.
- **axes - int[axis0, axis1, axis2]**:
    - Default value is [1].
      Shape inference supports negative axes; the NPU subgraph implementation primarily supports [1] and [1, 2, 3].
- **eps - float**:
    - Default value 1e-5, used to avoid division by zero.
- **std_multipler - float**:
    - Only used by gaussian_topk; default value is 1.0.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

---

exSoftmax

Applies softmax normalization along the specified axis of the input; the output shape is the same as the input.

FORMULA

`exSoftmax(input, axis) = exp(input - max(input, axis)) / sum_axis(exp(input - max(input, axis)))`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::softmax`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Subject to the constraints of transpose specifications.

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **axis - int**:
  - The axis for softmax; 1 is the channel direction, 3 is the width direction. Supporting only axis 1 and 3 is an RKNN platform limitation; the ONNX standard allows any valid axis.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exSoftmaxMask

Masked Softmax computation: sets masked positions to -inf using mask_value, then applies Softmax.

FORMULA

`input(x, mask_t, mask_value) = x + (mask_t - 1) * inf, if mask_value = 0`
`x + mask_t * (-inf), if mask_value = 1`

`exSoftmaxMask(input, axis) = exp(input - max(input, axis)) / sum_axis(exp(input - max(input, axis)))`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::softmax`

INPUTS

- **input** - `T1` (required):
  - Shape: NCHW[batch, channel, height, width]
  - Subject to the constraints of transpose specifications.
- **mask** - `T1` (optional):
  - Shape: NCHW[1, channel, height, width]
  - channel range: [1, 8192]
  - width range: [1, 8192]
  - When axis=1, width has no restriction; when axis=3/-1, width range is [1, 8192].

OUTPUTS

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **axis - int**:
  - The axis for softmax; 1 is the channel direction, 3 is the width direction. Supporting only axis 1 and 3 is an RKNN platform limitation; the ONNX standard allows any valid axis.
- **mask_value - int**:
  - 0 indicates positions to keep, 1 indicates positions to mask.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>`):

Matrix Operations

exMatMul

Matrix multiplication operation.

FORMULA

`Y = A · B + C, where C is bias (bias is optional)`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::matmul`, `aten::bmm`, `aten::mm`

INPUTS

- **A** - `T1` (required):
  - Shape: NK1N[batch, k, 1, n]
  - Quantization: per-layer、per-channel、group32/64/128
- **B** - `T1` (required):
  - Shape: NK1M[batch, k, 1, m]
  - Quantization: per-layer
- **C** - `T1, T2` (optional):
  - Shape: NM1M[batch, n, 1, m]
  - Quantization: per-layer
  - Supports NumPy-style broadcasting; broadcast constraints are the same as the Expand operator.

OUTPUTS

- **Y** - `T1`:
  - Shape: NM1M[batch, n, 1, m]
  - Quantization: per-layer

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`const_tensor<float16>`):

---

exMatMulActivation

Fused exMatMul and generic Activation computation; activation type is determined by the Activation sub-operator.

FORMULA

`output = Activation(exMatMul(input, weight, bias))`

INHERITANCE

Inherits from exMatMul.

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::matmul`, `aten::bmm`, `aten::mm`

LLM Extension Operators

exAttention

General attention subgraph operator supporting FullAttention and SlidingAttention modes.

FORMULA

`output = Matmul(Softmax(Matmul(rope(query, cos_pos_emb, sin_pos_emb), rope(key, cos_pos_emb, sin_pos_emb)) + attention_mask), value)`
`output = Matmul(Softmax(Matmul(query, key) + attention_mask), value)`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::multi_head_attention_forward`

INPUTS

- **query** - `T1` (required):
  - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]
- **key** - `T1` (required):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **value** - `T1` (required):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **attention_mask** - `T1` (required):
  - Shape: NCHW[1, seq_len, 1, seq_len]
- **cos_pos_emb** - `T1` (optional):

    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
- **sin_pos_emb** - `T1` (optional):
    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
- **Th** - `T2` (required):
    - Shape: N[1]
    - history token len; the number of tokens previously processed.
- **Tc** - `T2` (required):
    - Shape: N[1]
    - current token len; the number of valid tokens in the current inference bucket.
- **Ts** - `T2` (required):
    - Shape: N[1]
    - system prompt token len; the aligned system prompt length. Both [182X, 3572] require alignment to a multiple of 32.
- **Tsr** - `T2` (required):
    - Shape: N[1]
    - Real system prompt token len; the actual system prompt length, less than or equal to Ts.

OUTPUTS

- **output** - `T1`:
    - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]

ATTRIBUTES

- **attention_type - string**:
    - Represents global attention and sliding window attention modes respectively. Valid values: [FullAttention, SlidingAttention], default FullAttention.
- **rope - int**:
    - Whether to use RoPE positional encoding; 0 means not used, 1 means used, default 0.
- **kvcache_store_method - string**:
    - Indicates the KV cache storage method; Normal stores float16 data directly, GroupQuant stores with group quantization. Valid values: [Normal, GroupQuant], default Normal.
- **kvcache_dtype - string**:
    - Indicates the KV cache data type. Valid values: [Float16, Int4_to_F8, Int4_to_F16]. Float16 uses float16 storage; Int4_to_F8 uses 4-bit integer quantization storage and converts to 8-bit float during computation; Int4_to_F16 uses 4-bit integer quantization storage and converts to 16-bit float during computation. Default Float16.

- **kvcache_buffer_len - int**:
  - Maximum length of KV cache, in tokens; must be less than the model's max_position_embeddings.
- **kvcache_group_size - int**:
  - Effective when kvcache_store_method is GroupQuant, indicating the group size for group quantization; when kvcache_store_method is Normal, defaults to 32.
- **kvcache_residual_depth - int**:
  - Effective when kvcache_store_method is GroupQuant, indicating that group quantization is triggered after storing this number of tokens; when kvcache_store_method is Normal, defaults to 32.
- **sliding_window_size - int**:
  - Effective when attention_type is SlidingAttention, indicating the sliding window size; when attention_type is FullAttention, defaults to -1.

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`):

---

exPagedAttention

An attention subgraph operator using a paged KV cache, supporting FullAttention, SlidingAttention, and multi-request Decode.

FORMULA

```
Q = rope(query, cos_pos_emb, sin_pos_emb) if rope == 1 else query
K_current = rope(key, cos_pos_emb, sin_pos_emb) if rope == 1 else key
K_all, V_all = concat(valid historical KV cache, valid K_current/value from the current iteration)
logits[h,q,k] = scale * dot(Q[h,q], K_all[kv_head(h),k]) + mask[q,k]
P[h,q,k] = exp(logits[h,q,k]) / (sum_j exp(logits[h,q,j]) + (exp(sinks[h]) if has_sink == 1 else 0))
output[h,q] = sum_k P[h,q,k] * V_all[kv_head(h),k]
```

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **query** - `T1` (required):
  - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]
  - num_attention_heads must be an integer multiple of num_key_value_heads. MHA, MQA, and GQA are supported.

- **key** - `T1` (required):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **value** - `T1` (required):
  - Shape: NCHW[num_key_value_heads, value_head_dim, 1, seq_len]
- **attention_mask** - `T1` (required):
  - Shape: NCHW[1, seq_len, 1, seq_len]
  - An additive mask between tokens in the current iteration. The valid historical-cache range and the SlidingAttention window mask are handled by the internal subgraph.
- **sinks** - `T3` (optional):
  - Shape: N[num_attention_heads]
  - When has_sink=1, this input immediately follows attention_mask. It must be an FP32 initializer and must also be declared as a graph input, as required by the current importer. There is one logit per query head, added only to the Softmax denominator. It is excluded from scale, RoPE, and mask processing and is not written to the KV cache. Subsequent inputs shift accordingly.
- **cos_pos_emb** - `T1` (optional):
  - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
  - Required when rope=1; placed after attention_mask or the optional sinks input.
- **sin_pos_emb** - `T1` (optional):
  - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
  - Required when rope=1; immediately follows cos_pos_emb.
- **control_variables** - `T2` (required):
  - Shape: NC[request_num, 9]
  - One row per request, with nine columns in this exact order: [Th, Tc, Ts, Tsr, longest_surviving_id, store_begin_id, Tr, Ti, request_masks].
    Th is the historical token count before the current iteration. Tc is the valid token count in the current iteration, at most bucket_size and equal to 1 during Decode.
    Ts is the aligned system-prompt length; it must be a multiple of 32 and less than kvcache_buffer_len. Tsr is the actual system-prompt length, satisfying 0 <= Tsr <= Ts. longest_surviving_id is the storage position of the oldest still-valid token in the main cache, or -1 for an empty cache. store_begin_id is the next write position in the main cache.
    Tr is the valid token count in the residual cache. Ti is the logical token position at the end of the previous iteration; it is compared with Th to handle rollback.
    In multi-request mode, request_masks controls Attention and KVStore for this request: 0 skips execution, and nonzero executes it. In single-request mode, it is treated as 1. longest_surviving_id, store_begin_id, Tr, and Ti are mutable state and must be carried forward with the corresponding KV cache for subsequent inference. Th/Tc/Ts/Tsr are supplied by the caller.
    A new request can be initialized with [0, Tc, Ts, Tsr, -1, 0, 0, 0, 1]. Cache state is reset internally when Th=0.

OUTPUTS

- **output** - `T1` :
  - Shape: NCHW[num_attention_heads, value_head_dim, 1, seq_len]

ATTRIBUTES

- **attention_type - string**:
  - Explicitly selects full-attention or sliding-window-attention mode.
- **scale - float**:
  - The scale factor for QK logits, defaulting to 1.0. For standard scaled dot-product attention, the caller must set it to 1/sqrt(head_dim).
- **has_sink - int**:
  - Defaults to 0. A value of 1 indicates a constant sinks input; supported in both FullAttention and SlidingAttention.
- **rope - int**:
  - Defaults to 0. A value of 1 applies RoPE internally to query/key and requires both cos_pos_emb and sin_pos_emb. Set to 0 when RoPE is applied externally.
- **kvcache_store_method - string**:
  - Specifies the KV-cache data type. Float16 stores float16 values. Int4_to_F8 uses 4-bit integer quantized storage and converts to 8-bit floating point for computation. Int4_to_F16 uses 4-bit integer quantized storage and converts to 16-bit floating point for computation. Defaults to Float16.
- **kvcache_dtype - string**:
  - Normal uses Float16. Other types are used for GroupQuant; the F8/F16 suffix indicates the precision used by the computation path. If unspecified, the compiler configuration is used.
- **kvcache_buffer_len - int**:
  - The maximum token capacity of the main KV cache per request; must be positive. If unspecified or set to -1, the compiler configuration is used.
- **tokens_per_page - int**:
  - The number of KV-cache tokens per page, defaulting to 512; must be positive and is capped at kvcache_buffer_len. FullAttention uses this page size; SlidingAttention uses a single page covering the entire kvcache_buffer_len.
- **kvcache_group_size - int**:
  - Effective when kvcache_store_method is GroupQuant, indicating the group size for group quantization; when kvcache_store_method is Normal, defaults to 32.
- **kvcache_residual_depth - int**:
  - Effective when kvcache_store_method is GroupQuant, indicating that group quantization is triggered after storing this number of tokens; when

kvcache_store_method is Normal, defaults to 32.

- **request_num - int**:
  - Defaults to 1. The actual request count is determined statically by dimension 0 of control_variables; this attribute must match that dimension.
- **sliding_window_size - int**:
  - The window size for SlidingAttention; ignored for FullAttention. Defaults to -1.

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`): **T3** in (`tensor<float>`):

---

exSDPAttention

Performs scaled dot-product attention with optional mask.

FORMULA

```
q = Query
k = Key
v = Value

score = MatMul(k, q) * scale
if Mask is not None:
    score = score + Mask

prob = Softmax(score, axis=1)
Y = MatMul(v, prob)
```

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::scaled_dot_product_attention`

INPUTS

- **Query** - `T1` (required):
  - Shape: NCHW[batch, query_length, head, query_channel]
  - Quantization: per-layer
  - q matrix input, corresponding to query in the code.
- **Key** - `T1` (required):
  - Shape: NCHW[batch, key_length, head, key_channel]
  - Quantization: per-layer

    - k matrix input, corresponding to key in the code; participates in qk_matmul.
- **Value** - `T1` (required):
    - Shape: NCHW[batch, value_length, head, value_channel]
    - Quantization: per-layer
    - v matrix input.
- **Mask** - `T1` (optional):
    - Shape: NCHW[batch, key_channel, head, query_channel]
    - Quantization: per-layer
    - attention mask; when present, it is added to the qk_matmul result before softmax. The shape must be broadcastable to the qk_matmul output.

OUTPUTS

- **SDPAOutput** - `T1`:
    - Shape: NCHW[batch, query_length, head, value_channel]
    - Quantization: per-layer
    - Final output of scaled dot-product attention.

ATTRIBUTES

- **scale - float**:
    - Default value 1.0, applied to the qk_matmul result.
- **QMatmul - string**:
    - Quantization parameter description for qk_matmul output.
- **QAdd - string**:
    - Quantization parameter description for mask add output; only used when Mask is present.
- **QSoftmax - string**:
    - Quantization parameter description for softmax output.

DATA TYPE CONSTRAINTS

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

---

exLinearAttention

Chunk Gated Delta Rule linear attention subgraph operator; computes recursive linear attention of query/key/value in chunks.

FORMULA

`output = LinearAttention(query, key, value, g, beta, initial_state)`

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **query** - `T1` (required):
  - Shape: HD1S[H, D, 1, S]
  - Quantization: per-layer
  - H is num_heads, D is head_dim, S is seq_len.
- **key** - `T1` (required):
  - Shape: HD1S[H, D, 1, S]
  - Quantization: per-layer
- **value** - `T1` (required):
  - Shape: HD1S[H, D, 1, S]
  - Quantization: per-layer
- **g** - `T1` (required):
  - Shape: 1H1S[1, H, 1, S]
  - Quantization: per-layer
  - Gate factor.
- **beta** - `T1` (required):
  - Shape: 1H1S[1, H, 1, S]
  - Quantization: per-layer
- **Tc** - `T2` (required):
  - Shape: S[seq_len]
  - Effective seq_len scalar. When Tc < seq_len, left-side zero padding is applied to query/key/value along the seq dimension.
- **initial_state** - `T1` (optional):
  - Shape: HDD[H, D, 1, D]
  - Quantization: per-layer
  - Optional initial recurrent state. If provided by the frontend, it is directly connected.

OUTPUTS

- **output** - `T1` :
  - Shape: HD1S[H, D, 1, S]

    - Quantization: per-layer
- **final_state** - `T1` :
    - Shape: HDD[H, D, 1, D]
    - Quantization: per-layer
    - Only output when OutputFinalState is true.

ATTRIBUTES

- **ChunkSize - int**:
    - Chunk size, default 32. seq_len must be a multiple of ChunkSize (padding is the caller's responsibility).
- **UseQkL2Norm - bool**:
    - Whether to apply L2 normalization to query/key; default false.
- **OutputFinalState - bool**:
    - Whether to output the final recurrent state; default false.

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>` , `tensor<bfloat16>`): **T2** in (`tensor<int32>`):

NOTES

Subgraph operator; the actual implementation is expanded by createSubGraph at the IR layer; emit only returns success.
The batch dimension is fixed to 1. seq_len must be padded to a multiple of ChunkSize before entering the subgraph.

---

exRoPE

General subgraph operator supporting RoPE positional encoding.

FORMULA

```
output = input * cos_pos_emb + rotate_half(input) * sin_pos_emb
```

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **input** - `T1` (required):
    - Shape: NCHW[batch_size, channel, 1, seq_len]
- **cos_pos_emb** - `T1` (required):

    - Shape: NCHW[batch_size, channel, 1, position_embeddings]
    - position_embeddings == 1 or
      seq_len <= position_embeddings <= max_position_embeddings。
- **sin_pos_emb** - `T1` (required):
    - Shape: NCHW[batch_size, channel, 1, position_embeddings]
    - position_embeddings == 1 or
      seq_len <= position_embeddings <= max_position_embeddings。
- **Th** - `T2` (optional):
    - Shape: N[1]
    - history token len; the number of tokens previously processed.
- **Tc** - `T2` (optional):
    - Shape: N[1]
    - current token len; the number of valid tokens in the current inference bucket.
- **Ts** - `T2` (optional):
    - Shape: N[1]
    - system prompt token len; the aligned system prompt length. Both [182X, 3572] require alignment to a multiple of 32.
- **Tsr** - `T2` (optional):
    - Shape: N[1]
    - Real system prompt token len; the actual system prompt length, less than or equal to Ts.

OUTPUTS

- **output** - `T1` :
    - Shape: NCHW[batch_size, channel, 1, seq_len]

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`):

NOTES

When exRoPE is used in an LLM model and
full cos/sin_pos_emb is provided (position embeddings > seq_len), inputs such as Th/Tc/Ts/Tsr will be present. exRoPE internally calculates the actual length of cos/sin_pos_emb to use based on Th/Tc/Ts/Tsr, thereby supporting dynamic sequence length inference. In other cases, seq_len == position_embeddings and
Th/Tc/Ts/Tsr do not exist.

---

exSum

Weighted sum of multiple inputs, commonly used for merging LoRA multi-branch results.

FORMULA

`Y = Sum(X_i * scale_i) for i in 0..N-1`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::sum`

INPUTS

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer
  - Variable-length input list; at least 1, all input shapes must be identical.

OUTPUTS

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - Quantization: per-layer

ATTRIBUTES

- **scales - float[scale_1, ..., scale_N-1]**:
  - Weighting coefficient for each extra input (except the first); the count is the number of inputs - 1.

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>`):

Type Conversion

Cast

Converts each element of the input tensor to the specified target data type.

FORMULA

`output = Cast(input, to)`

OPERATOR LEVEL

Standard ONNX Operator

PYTORCH SOURCE

`aten::to`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **to - int**:
  - Target data type, mapped from ONNX to_type; supports bool/int8/uint8/int16/int32/float16/float32.

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`): **T2** in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`):

---

exDataConvert

Converts the data type of the input tensor to the specified target type.

FORMULA

`output = exDataConvert(input, input_type, output_type)`

OPERATOR LEVEL

RKNN3 Extension Operator

PYTORCH SOURCE

`aten::to`

INPUTS

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

ATTRIBUTES

- **input_type - int**:
  - Input data type.
- **output_type - int**:
  - Output data type.

DATA TYPE CONSTRAINTS

T1 in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`): **T2** in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`):

I/O Processing

exInputProc

Performs input data preprocessing.

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **B** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<float32>` , `tensor<float16>` , `tensor<bool>` , `tensor<uint8>`): **T2** in (`tensor<float16>` , `tensor<int8>`):

---

exOutputProc

Performs output data post-processing.

OPERATOR LEVEL

RKNN3 Extension Operator

INPUTS

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

OUTPUTS

- **B** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

DATA TYPE CONSTRAINTS

T1 in (`tensor<float16>` , `tensor<int8>`): **T2** in (`tensor<float32>` , `tensor<float16>` , `tensor<bool>` , `tensor<uint8>`):